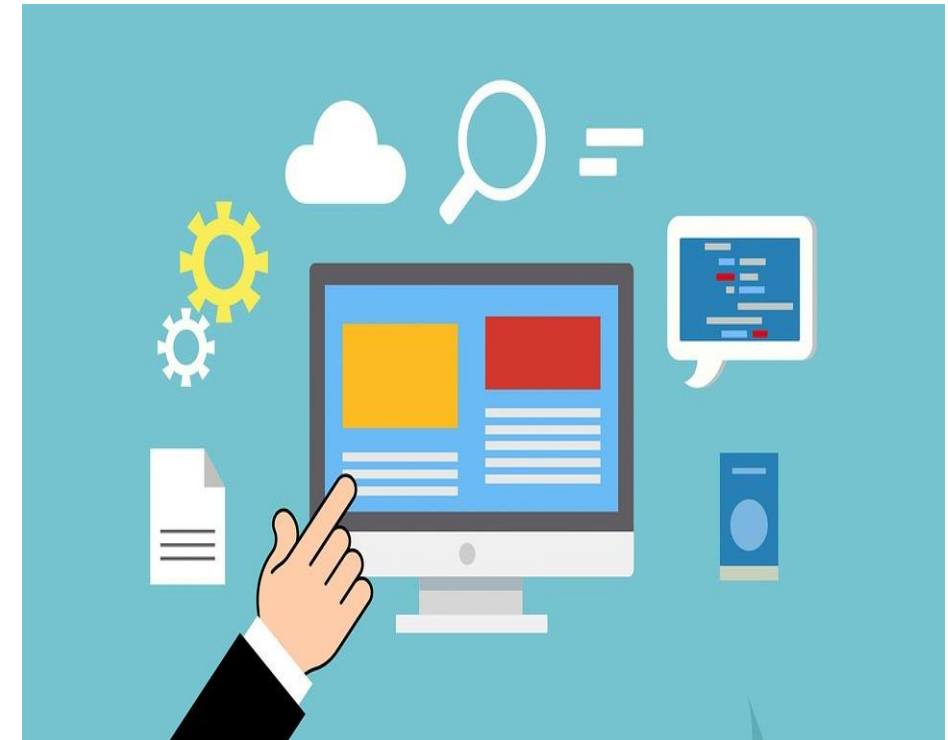


DESARROLLO DE SISTEMAS WEB

Licenciatura en Ingeniería de Ciberseguridad e
Infraestructura de Cómputo

```
else if (i==2)
{
    var atpos=inputs[i].getAttribute("atpos");
    var dotpos=inputs[i].getAttribute("dotpos");
    if (atpos<1 || dotpos>atpos)
        document.getElementById("error").innerHTML+=
        "else
        document.getElementById(divi)";
}
```



Fuentes de información

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- Shklar Leon y Rosen Richard. Web Application, Principles, Protocols and Practices. Wiley. 2009.
- Gasston, Peter. The Modern Web: Multi-Device Web Development with HTML5, CSS3, and JavaScript. No Starch Press. 2013.
- Jennifer Niederst Robbins. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics. O'REILLY. 2012.
- Buschmann, Meunier, Rohnert, Sommerlad, Stal, Pattern Oriented Software Architecture: A System of Patterns, Wiley, 1996.

Taxonomía de las aplicaciones web

Unidad 4. Desarrollo de Aplicaciones web dinámicas

Saberes teóricos

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- **Unidad 4. Desarrollo de Aplicaciones web dinámicas**
 - **Taxonomía de las aplicaciones web**
 - Marcos de trabajo (frameworks) para el desarrollo de aplicaciones web
 - Aplicaciones web para dispositivos móviles
 - Servidores web
 - Estructura de las aplicaciones web
 - Modelos de capas
 - Separación de estructura, presentación y comportamiento
 - Entrega de contenido multimedia en la web

¿Qué es una aplicación web?

Unidad II. Frameworks de desarrollo web

- Se denomina **aplicación web** o **software web** a un software que los usuarios pueden utilizar accediendo a un servidor web a través de **internet** o de una **intranet** mediante un navegador.
- Cuando hablamos de **software web** nos referimos a aplicaciones que se instalan en servidores que son ordenadores dedicados a proveer servicios, realizar tareas, ante peticiones de otros ordenadores.
- Estas aplicaciones web o softwares web son desarrollados por programadores que utilizan entornos de programación **frontend** y **backend**.
- Los datos y la lógica de negocio quedan centralizados en el propio servidor y los usuarios podrán acceder a ellos por medio de sus propios navegadores web.



Taxonomía de las aplicaciones web

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- Las aplicaciones web pueden clasificarse en varios tipos de acuerdo a sus características:
 - Aplicaciones web estáticas.
 - Aplicaciones web dinámicas.
 - Aplicaciones web de comercio electrónico.
 - Aplicaciones web de gestión de contenido.
 - Aplicaciones web de página única.
 - Aplicaciones web de portal.
 - Aplicaciones web progresivas.

Aplicaciones web estáticas

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- Las aplicaciones web estáticas son los sitios diseñados para presentar información a los visitantes sin permitir que interactúen con el contenido más allá de su lectura. Estos sitios son de los más simples que existen y generalmente están diseñados únicamente como datos HTML con algunas líneas de código CSS.
- En ocasiones incluyen imágenes o videos integrados, aunque por convención se piensa que deben ser lo más simples posibles para diferenciarlos de otros tipos de aplicaciones web.
- Sitios de perfil o currículum.

Aplicaciones web dinámicas

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- Las aplicaciones web dinámicas, como su nombre lo indica, son sitios en la red que no se mantienen estáticos a lo largo del tiempo y que, por el contrario, están en un proceso constante de actualización que hace que haya diferentes contenidos cada vez que se visitan.
- Si los sitios web estáticos destacan por permanecer idénticos todo el tiempo, los dinámicos facilitan la tarea de actualizarlos manualmente, al estar acompañados de bases de datos que ofrecen información constante y en tiempo real. Pero para ello requieren de una mayor programación, tanto con HTML y CSS como con PHP, JavaScript o ASP.
- Redes sociales.

Aplicaciones web de comercio electrónico

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- Un sitio web de comercio electrónico debe estar optimizado para muchas cosas: mostrar un producto, describirlo y exponer sus características principales; permitir que se añadan o quiten productos; administrar el pago de los clientes; generar facturas electrónicas y hasta contar con un buen diseño y herramientas de contacto como formularios y chat en vivo.
- Por otro lado, un buen sitio web de comercio electrónico debe ser al mismo tiempo una aplicación web dinámica. Esto se debe a que aquellas marcas que venden productos en línea deben mantener siempre actualizada la disponibilidad de su mercancía, los costos y la información de consulta.
- Mercado libre, Amazon.

Aplicaciones web de gestión de contenido

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- Los CMS, también conocidos como sistemas de gestión de contenidos, son un tipo de aplicación web accesible desde navegador. Sirven para administrar los contenidos que se muestran en un sitio web, así como en cada una de sus páginas.
- Con estas herramientas es realmente fácil editar textos para blogs, integrar imágenes y videos dentro de una publicación, o incluso editar tus páginas en ventanas de desarrollo web con HTML o CSS.
- Como tal, estas aplicaciones funcionan como intermediarios entre tú y tu sitio web. Por ello están optimizadas con todas las herramientas necesarias para que tus contenidos estén actualizados y en la mejor condición para ser publicados en tu sitio.
- Wordpress, Drupal, Joomla

Aplicaciones web de página única

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- Si bien es común que las aplicaciones web y los sitios resguarden una importante cantidad de páginas, existen algunas otras que únicamente operan mediante una página en constante actualización.
- Esto significa que todas las acciones que realizas en un sitio se efectúan en la misma ventana. Aunque esto pueda parecer una tarea muy compleja —debido al tiempo de carga y a los volúmenes de información—, lo cierto es que una vez que el sitio ha cargado la navegación es rápida, eficiente y simple.
- Algunas redes sociales operan de este modo. En ellas puedes notar que los mismos iconos del menú se mantienen en el mismo lugar, sin importar a qué sección accedas.
- Gmail, Facebook.

Aplicaciones web de portal

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- Otro formato de aplicación web es aquella en la que, en contraposición a los sitios web de una sola página, un mismo portal funciona como matriz que redirige al visitante, mediante enlaces, a otras páginas. Este tipo de aplicaciones son especialmente útiles cuando un mismo sitio resguarda información o funcionalidades de muchos tipos y que requieren ser clasificados en categorías fácilmente accesibles.
- Si, por ejemplo, una empresa ofrece una gran cantidad de servicios, lo mejor es contar con una página que funcione como portal de acceso a las múltiples secciones del sitio.
- Algunas de estas páginas pueden, además, estar limitadas para únicamente ser accesibles a ciertos usuarios o suscriptores.
- Universidad Veracruzana.

Aplicaciones web progresivas

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- Desde hace algunos años se ha notado una clara tendencia para migrar las aplicaciones web a formatos móviles. Muchas de las herramientas que hasta hace una década podíamos utilizar desde un sitio web han cambiado al formato de aplicación, que solo puede descargarse desde las tiendas oficiales de apps.
- Para mitigar este fenómeno, algunas aplicaciones móviles han optado por integrar la experiencia de navegación y uso de las plataformas mediante interfaces que se ajustan a la experiencia de uso móvil. Es por ello que son conocidas como aplicaciones web progresivas.
- Esto ayuda a que las marcas sigan promoviendo su identidad gráfica y de navegación, ya sea que ocurra dentro de un navegador, una aplicación web o un software instalado.
- Spotify, Netflix, HBO, Youtube Music.

Marcos de trabajo (frameworks) para el desarrollo de aplicaciones web

Unidad 4. Desarrollo de Aplicaciones web dinámicas

Saberes teóricos

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- **Unidad 4. Desarrollo de Aplicaciones web dinámicas**
 - Taxonomía de las aplicaciones web
 - Marcos de trabajo (frameworks) para el desarrollo de aplicaciones web
 - Aplicaciones web para dispositivos móviles
 - Servidores web
 - Estructura de las aplicaciones web
 - Modelos de capas Separación de estructura, presentación y comportamiento
 - Entrega de contenido multimedia en la web

Introducción

Unidad II. Frameworks de desarrollo web

- El uso de frameworks es una de las técnicas más usadas por los programadores para el **desarrollo web**.
- No es para menos, pues este tipo de software brinda un sinnúmero de facilidades para centrarse en la lógica del funcionamiento del sitio, en lugar de gastar el tiempo en la escritura de códigos que se pueden reutilizar.
- Se trata de soluciones versátiles y funcionales que han facilitado la tarea de la programación.
- Son módulos que ayudan en la configuración de funciones y bloques dentro del **desarrollo de aplicaciones web**.

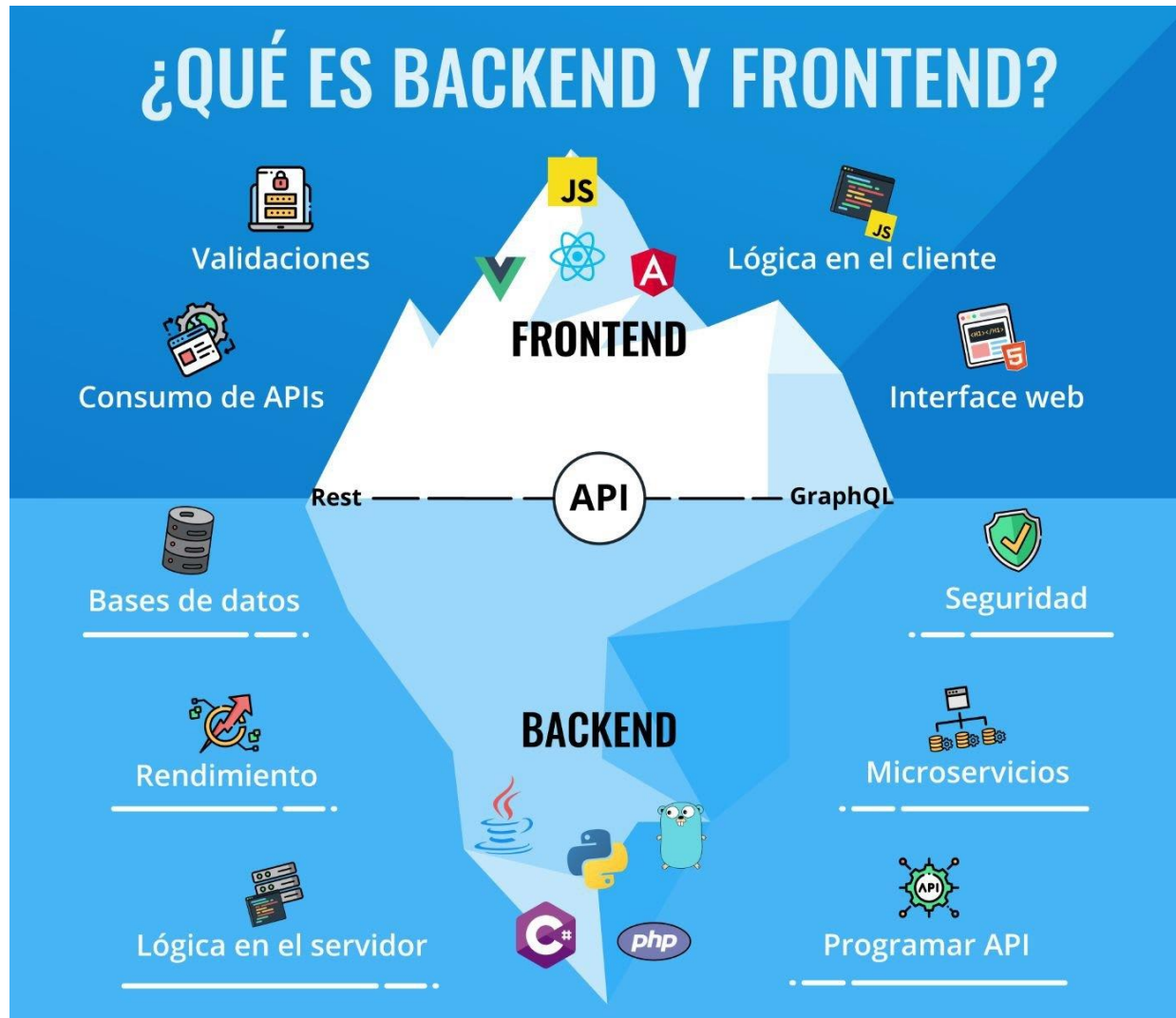
Página web, sitio web, aplicación web

Unidad II. Frameworks de desarrollo web



Partes de una aplicación Web

Unidad II. Frameworks de desarrollo web



Partes de una aplicación Web

Unidad II. Frameworks de desarrollo web

Frontend	Backend
Se diseña en lenguajes como HTML, CSS y JavaScript	Se programa en lenguajes como PHP, Python y C+
Prioriza la experiencia del usuario	Prioriza las necesidades de la marca
Centrado en asegurar una buena experiencia de navegación	Centrado en asegurar un buen desempeño del sitio
Optimiza la experiencia desde los navegadores	Optimiza la experiencia desde los servidores
Sus resultados definen el estilo del sitio y siempre son visibles	Sus resultados dan sostén al sitio, pero no son accesibles al usuario
Recorre al diseño de contenidos como textos, imágenes o multimedia	Requiere el diseño y uso de herramientas como bases de datos y hardware
Requiere conocimientos de diseño	Precisa de conocimientos de lógica
Sus profesionales necesitan una amplia capacidad creativa	Sus profesionales necesitan una gran capacidad analítica

El camino del desarrollo web frontend

Unidad II. Frameworks de desarrollo web

CONVIÉRTETE EN UN FRONTEND PROFESIONAL

POR LEONIDAS ESTEBAN DE  Platzi



GIT Y GITHUB



PROGRAMACIÓN
BÁSICA



DESARROLLO WEB



FUNDAMENTOS
DE JAVASCRIPT



RESPONSIVE
DESIGN



LUEGO ESPECIALÍZATE EN:



REACT CON REDUX



VUE JS



ANGULAR



El camino del desarrollo web backend

Unidad II. Frameworks de desarrollo web

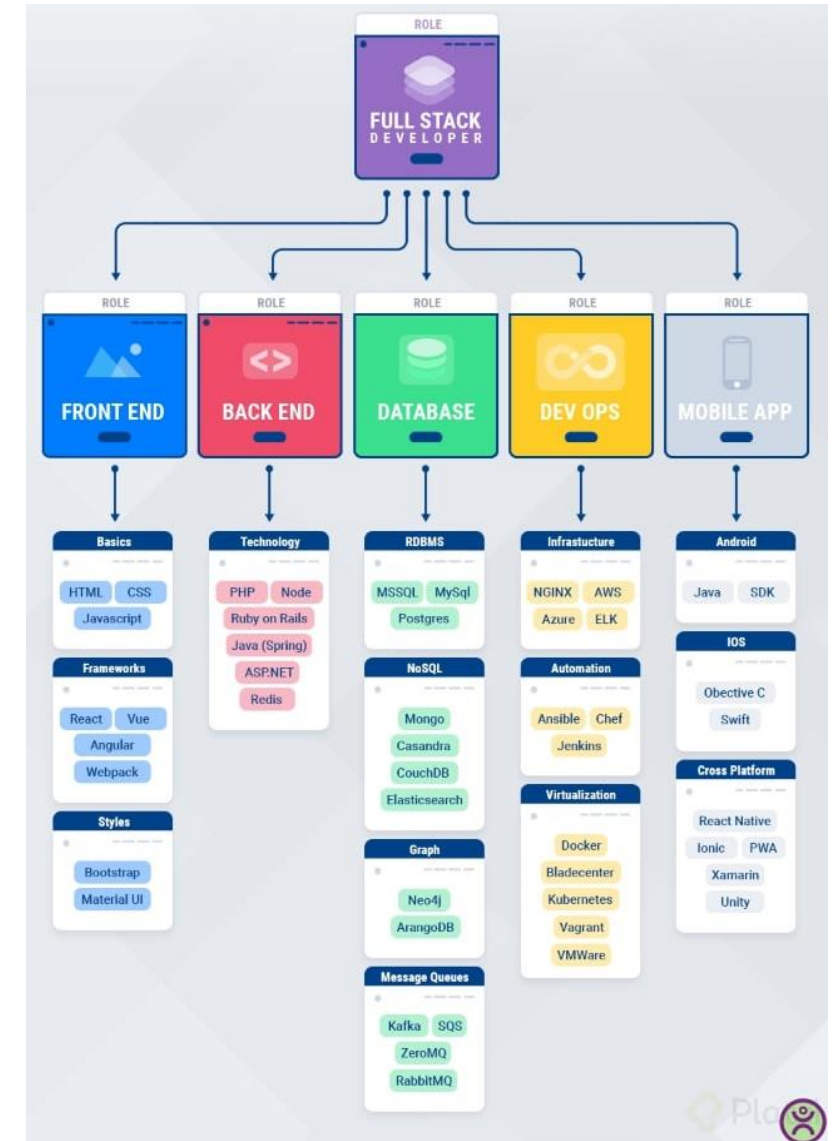
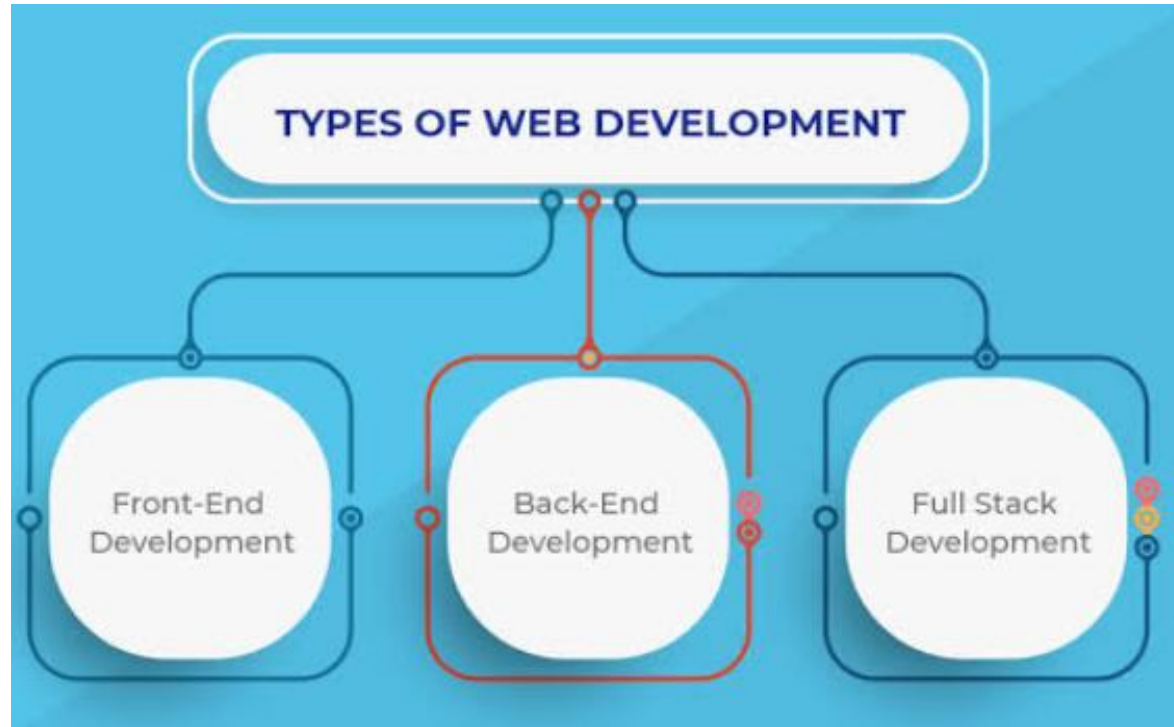
QUÉ NECESITAS PARA SER BACKEND PROFESIONAL

POR DAVID TOCA DE  Platzi



Full Stack Developer

Unidad II. Frameworks de desarrollo web



¿Qué es un framework de desarrollo?

Unidad II. Frameworks de desarrollo web

- Un framework es una estructura o un **conjunto de herramientas predefinidas** que proporciona una base para la construcción de aplicaciones.
- Está diseñado para ayudar a los desarrolladores a crear aplicaciones de manera más eficiente y consistente, al proporcionar una serie de componentes y funciones comunes que se utilizan con frecuencia en el desarrollo.



¿Qué es un framework de desarrollo web?

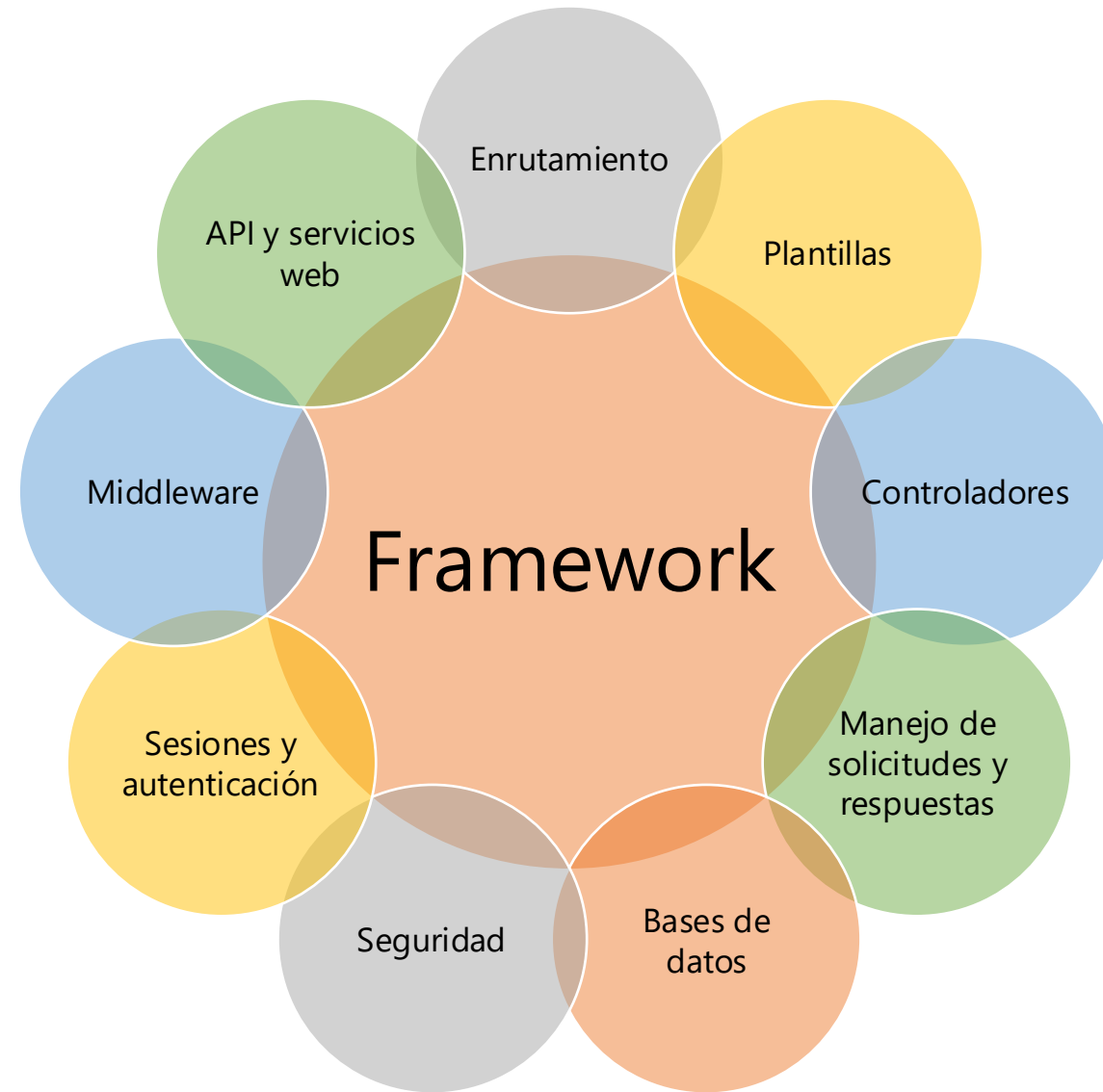
Unidad II. Frameworks de desarrollo web

- Un **framework para aplicaciones web** es un framework diseñado para **apoyar el desarrollo de sitios web**, sistemas web y servicios web.
- Este tipo de frameworks **intenta aliviar el exceso de carga** asociado con actividades comunes usadas en desarrollos web.
- Típicamente, puede incluir soporte de programas, bibliotecas, y un lenguaje interpretado, entre otras herramientas, para así ayudar a desarrollar y unir los diferentes componentes de un proyecto.



Características de los frameworks

Unidad II. Frameworks de desarrollo web



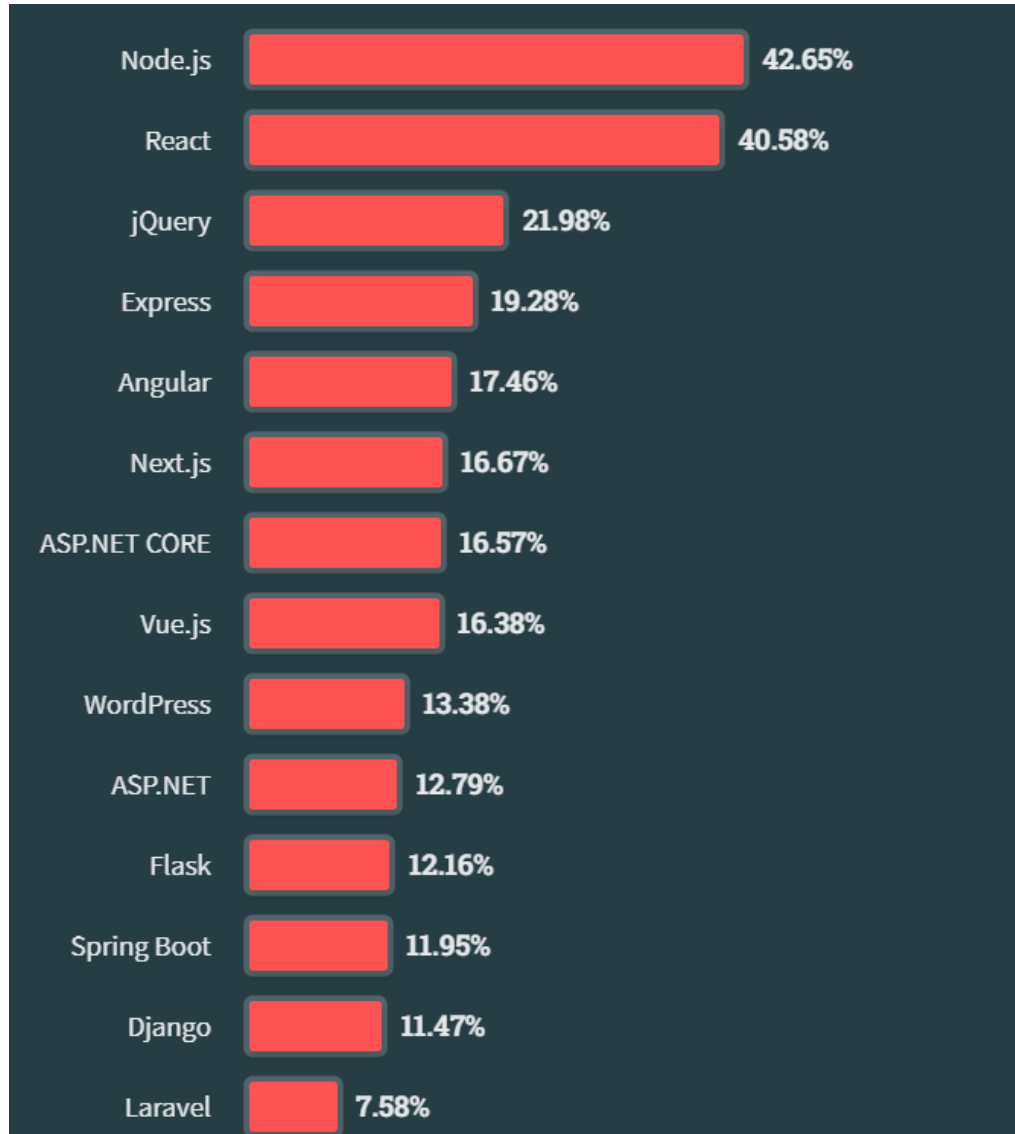
Características de los frameworks

Unidad II. Frameworks de desarrollo web

- **Enrutamiento:** suelen proporcionar un sistema de enrutamiento que facilita la gestión de las URL y las rutas de las páginas dentro de una aplicación.
- **Plantillas:** permiten crear un sitio web de manera más sencilla mediante el uso de plantillas predefinidas o motores de plantillas que ayudan a generar el código HTML de manera dinámica.
- **Controladores:** a menudo utilizan el patrón de diseño MVC (modelo-vista-controlador) o una variante de él, lo que significa que facilitan la separación de la lógica de negocio (controlador), de la presentación (vista) y los datos (modelo).
- **Manejo de solicitudes y respuestas:** ofrecen funciones para gestionar solicitudes HTTP entrantes y generar respuestas HTTP, lo que simplifica la interacción con el cliente.
- **Bases de datos:** muchos entornos integran herramientas y abstracciones para interactuar con bases de datos, lo que facilita el almacenamiento y recuperación de los mismos.
- **Seguridad:** suelen incluir mecanismos de seguridad incorporados para proteger contra amenazas tales como los ataques de inyección SQL o ataques de scripting entre sitios (XSS).
- **Sesiones y autenticación:** proporcionan funciones para gestionar las sesiones de usuario y su autenticación, lo que facilita la creación de sistemas de inicio de sesión y control de acceso.
- **Middleware:** permiten la inclusión de componentes de esta clase que pueden realizar tareas adicionales como la compresión de respuestas, el registro de solicitudes o la autenticación en el flujo de la aplicación.
- **API y servicios web:** facilitan la creación y consumo de estas interfaces y servicios, lo que permite la integración con otras aplicaciones y servicios.

Frameworks y tecnologías web más populares

Tecnologías web



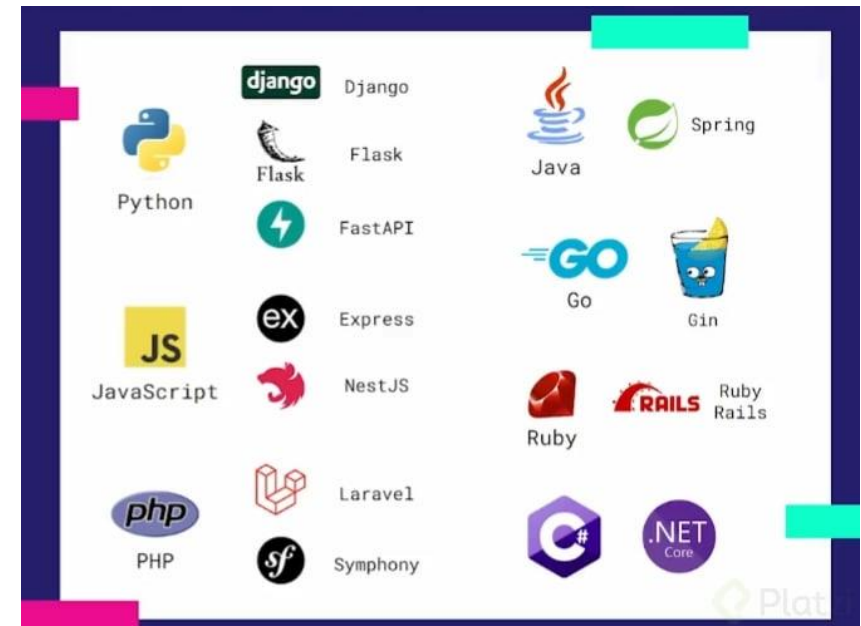
<https://insights.stackoverflow.com/survey>



Introducción

Tipos de frameworks existentes

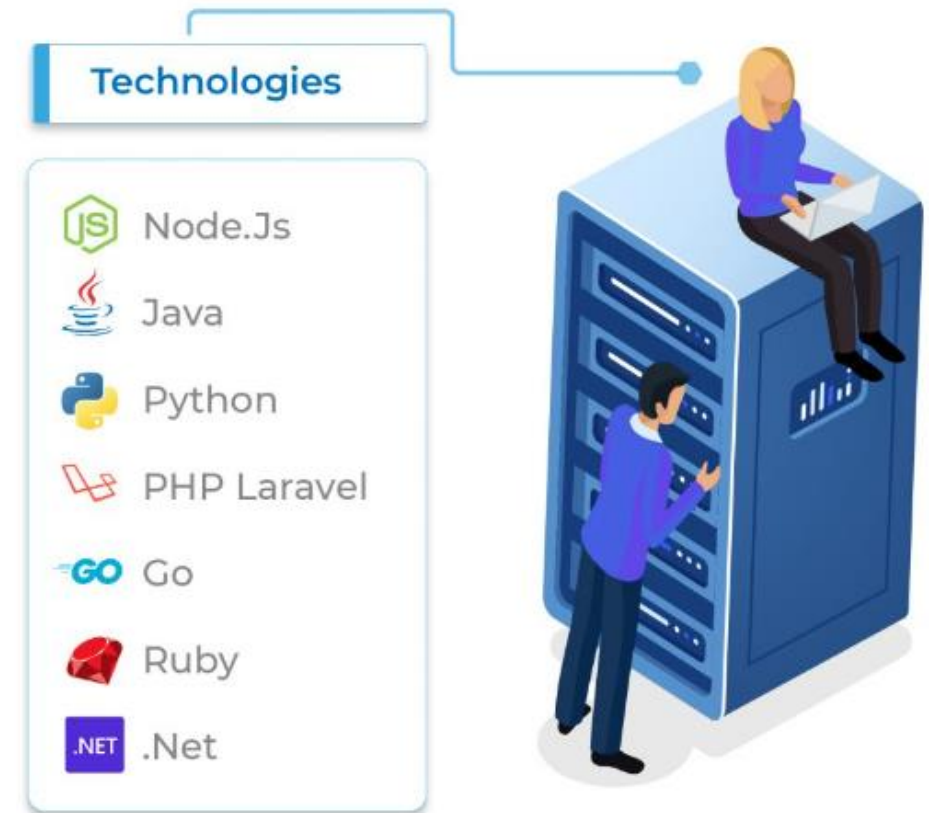
- La elección del framework a menudo **depende** del lenguaje de programación preferido, los requisitos del proyecto y las preferencias del desarrollador.
- Solo hay que tomar a consideración que la utilización de un framework en el desarrollo de una aplicación implica un cierto **coste** inicial de **aprendizaje**.



Tipos de frameworks existentes

Tipos de frameworks existentes

- Los tipos de frameworks se dividen principalmente en dos tipos:
 - Frameworks para **frontend**.
 - Frameworks para **backend**.



Tipos de framework para desarrollo web

Tipos de frameworks existentes

- **Frameworks de frontend**

- Frameworks de JavaScript
 - React
 - Vue.js
 - Angular
 - jQuery (jQuery UI)
 - Ember
 - Backbone.js
- Frameworks de CSS
 - Bootstrap
 - Tailwind CSS
 - Bulma
 - Foundation
 - Semantic UI
 - Materialize

- **Frameworks de backend**

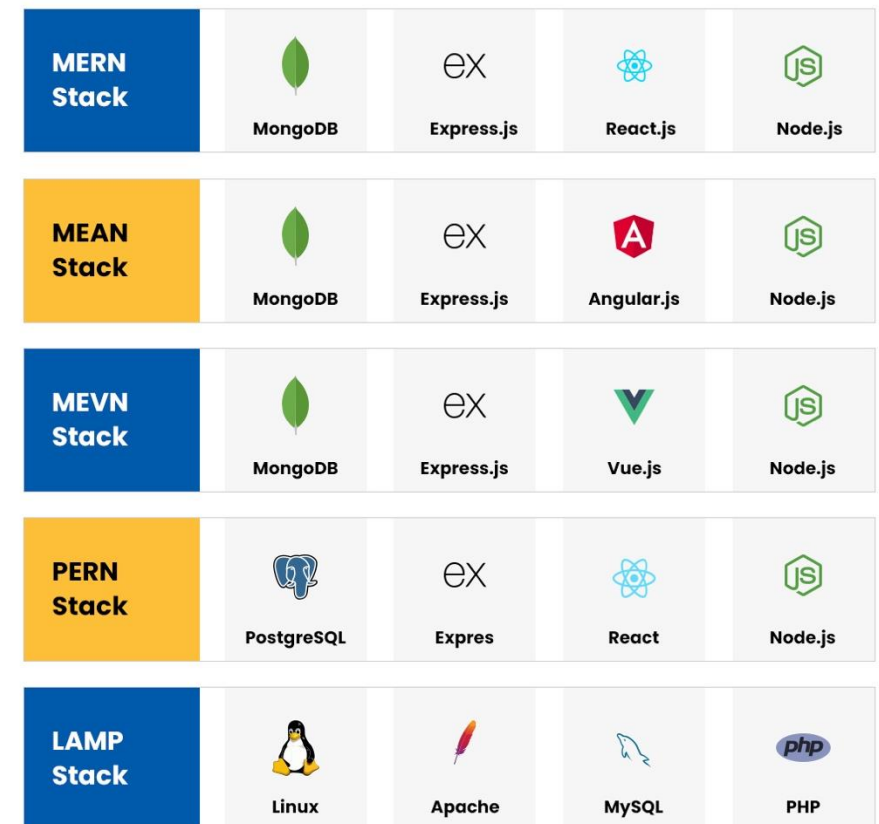
- ExpressJS (NodeJS)
- Django, Flask (Python)
- Spring Boot, Java Server Faces (Java)
- Ruby on Rails, Sinatra (Ruby)
- ASP.NET Core, Entity Framework Core (C#)
- Laravel, Slim, Symfony (PHP)

Combinando estas tecnologías se puede armar un **stack** de programación web.

Web Development Stack

Tipos de frameworks existentes

- Consta de varios de los mejores marcos para el desarrollo web, lenguajes de programación **frontend**, lenguajes de programación **backend**, servidores y herramientas de desarrollo que ayudan a crear aplicaciones web.
- **Web Stack** también es conocido con otros nombres como Solution Stack, Data Ecosystem, o Technology Infrastructure.
 - **MERN** – Best Stack for React.js
 - **MEAN** – Best Stack for Node.js
 - **MEVN** – Best stack for Vue.js
 - **PERN** – Best Stack for PostgreSQL
 - **LAMP** – Best Stack for WordPress
 - **Ruby On Rails** – Best *Full Stack* Stack for Ruby
 - **Django** – Best *Full Stack* for Python
 - **NET** – Best *Full Stack* for Microsoft
 - **JAMSTACK** – Best for Static Sites



Tipos de framework para desarrollo web

Frameworks de backend

- **Frameworks de backend**

- Express JS (Node JS)
- Django, Flask (Python)
- Spring Boot, Java Server Faces (Java)
- Ruby on Rails (Ruby)
- ASP.NET Core (C#)
- Laravel, Slim, Symfony (PHP)

Django (Python)

Frameworks de backend



- Lanzado en **2005**. Django es un framework de desarrollo web de código abierto, escrito en Python, que respeta el patrón de diseño conocido como MVC. Ofrece una versión solo .
- **Características**
 - Proporciona un ORM (Object Relational Mapping) que permite a los desarrolladores interactuar con la base de datos utilizando objetos de Python en lugar de escribir consultas SQL de forma directa.
 - Ofrece herramientas integradas para gestionar la autenticación de usuarios, proteger contra ataques comunes como la inyección SQL y el cross-site scripting (XSS), además de gestionar permisos y autorización.
 - Utiliza un sistema de enrutamiento y vistas que facilita la definición de URL.
 - Proporciona un sistema de plantillas que permite definir la presentación de manera separada de la lógica de la aplicación, lo que promueve una estructura limpia y mantenible.
 - Facilita la internacionalización de aplicaciones, lo que posibilita que las aplicaciones sean utilizadas en múltiples idiomas y regiones.
 - Ofrece protección contra vulnerabilidades comunes como CSRF (cross-site request forgery) y XSS.
 - Django REST Framework es un set de herramientas poderoso y flexible para la construcción de APIs.
 - Instagram, Pinterest, The New York Times.
- **¿Cuándo utilizar?**
 - Todo tipo de proyectos web usando el servidor web Gunicorn o Apache.

Flask (Python)

Frameworks de backend



- Lanzado en **2010**. Flask es un framework minimalista escrito en Python que permite crear aplicaciones web rápidamente y con un mínimo número de líneas de código. Se trata de un microframework con diseño minimalista y enfocado en el rendimiento.
- **Características**
 - Flask se llama a sí mismo un «microframework» porque proporciona solo las herramientas esenciales para el desarrollo web.
 - Facilita la definición de rutas y vistas para manejar solicitudes HTTP.
 - Se integra con el motor de plantillas Jinja2, que te permite crear vistas dinámicas y reutilizables.
 - Es altamente extensible. Puedes agregar funciones adicionales utilizando extensiones de Flask.
 - Es compatible con una variedad de bases de datos, incluyendo SQLite, PostgreSQL, MySQL y más.
 - Ofrece extensiones como Flask-RESTful para crear API REST de manera sencilla.
 - Está diseñado para el desarrollo rápido de aplicaciones. Puedes crear prototipos en corto tiempo y agregar características según las necesidades de tu proyecto.
 - Pinterest, twilio, netflix, lyft, zillow.
- **¿Cuándo utilizar?**
 - Todo tipo de proyectos web usando el servidor web Gunicorn o Apache.

Spring Boot (Java)

Frameworks de backend



Spring Boot®

- Lanzado en **2014**. Es un framework desarrollado para el trabajo con Java como lenguaje de programación. Se trata de un entorno de desarrollo de código abierto y gratuito.
- **Características**
 - Permite crear aplicaciones autónomas que se ejecutan por sí solas, sin depender de un servidor web externo, integrando un servidor web como Tomcat o Netty en la app durante el proceso de inicialización.
 - Creado para aplicaciones web y microservicios.
 - Es una versión extendida del framework Spring que ayuda a reducir el tiempo de desarrollo.
 - Alternativa más simple al módulo Spring MVC para crear aplicaciones web.
 - Las aplicaciones creadas con Spring Boot están preparadas para producción.
- **¿Cuándo utilizar?**
 - Aplicaciones escritas en Java utilizando servidores web Tomcat o Netty.

Java Server Faces (Java)

Frameworks de backend



- Lanzado en **2016**. JavaServer Faces es una tecnología y framework para aplicaciones Java basadas en web que simplifica el desarrollo de interfaces de usuario en aplicaciones Java EE. JSF usa JavaServer Pages como la tecnología que permite hacer el despliegue de las páginas web.
- **Características**
 - Una clara separación entre vista y modelo.
 - Desarrollo basado en componente, no en peticiones.
 - Las acciones del usuario se ligan muy fácilmente al código en el servidor.
 - Creación de familias de componentes visuales para acelerar el desarrollo.
 - JSF se integra dentro de la página JSP y se encarga de la recogida y generación de los valores de los elementos de la página.
- **¿Cuándo utilizar?**
 - Aplicaciones escritas en Java utilizando servidores web Tomcat, Glassfish.

ASP.NET Core (C#)

Frameworks de backend



- Lanzado en **2003**. Es un marco web multiplataforma desarrollado por Microsoft, de alto rendimiento y de código abierto. ASP.NET Core se puede utilizar para crear aplicaciones web, móviles, aplicaciones de IoT y backends. El marco está construido sobre el tiempo de ejecución de .NET Core, lo que facilita la implementación de aplicaciones en una variedad de entornos.
- **Características**
 - Las aplicaciones ASP.NET Core se pueden implementar en IIS , Azure o cualquier otro proveedor de alojamiento.
 - Un modelo de programación unificado que combina MVC y Web API.
 - Un sistema de enrutamiento flexible que se puede utilizar para definir rutas de aplicaciones.
 - Contenedor de inyección de dependencias integrado.
 - Se puede utilizar una variedad de componentes de middleware integrados para agregar funcionalidad a las aplicaciones, como autenticación, almacenamiento en caché y manejo de errores.
 - GettyImages, TacoBell, StackOverflow.
- **¿Cuándo utilizar?**
 - Todo tipo de aplicaciones desde pequeñas hasta empresariales.

Ruby on Rails (Ruby)

Frameworks de backend



- Lanzado en **2004**. Ruby on Rails se originó en Japón y fue construido como un marco web de propósito general. Permite a los programadores utilizar el lenguaje llamado Ruby, lo que lo hace muy fácil de aprender. El entorno de desarrollo estándar incluye una base de datos integrada llamada SQLite que se puede utilizar sin necesidad de una instalación independiente.
- **Características**
 - Está diseñado para facilitar la programación de aplicaciones web al hacer suposiciones sobre lo que cada desarrollador necesita para comenzar.
 - Incluye todo lo necesario para crear una aplicación web MVC respaldada por una base de datos, con el lenguaje de programación Ruby y el marco de aplicación web Rails.
 - Utiliza el patrón Active Record para mapear objetos de base de datos a modelos de Ruby.
 - Tiene un enrutador incorporado que facilita la definición de rutas y la gestión de URL.
 - Rails también es una gran herramienta para crear servicios web y API.
 - Twitter , GitHub y Shopify.
- **¿Cuándo utilizar?**
 - Su enfoque en la eficiencia y la productividad lo hace ideal para startups y proyectos que requieren entregas rápidas.

Laravel (PHP)

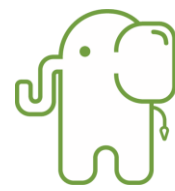
Frameworks de backend



- Lanzado en **2011**. Es un marco de desarrollo web de código abierto para PHP que me parece poderoso y elegante. Se ha convertido en uno de los frameworks PHP más populares y utilizados en la industria. Laravel se destaca por su enfoque en la elegancia del código, la simplicidad y la productividad en el desarrollo de aplicaciones.
- **Características**
 - Utiliza una sintaxis limpia y expresiva que hace que el código sea fácil de leer y escribir. Usa el patrón de diseño MVC.
 - Incluye Eloquent, un ORM (mapeo objeto-relacional) que facilita la interacción con bases de datos SQL. Este permite definir modelos de datos como clases de PHP y realizar consultas de base de datos utilizando una sintaxis orientada a objetos.
 - Proporciona un sistema de enrutamiento potente y fácil de usar que define rutas de manera clara y asigna controladores a acciones específicas.
 - Blade es el motor de plantillas incluido en Laravel que posibilita crear vistas de manera eficiente y reutilizable. Incluye características como herencia de plantillas y directivas que facilitan la creación de vistas dinámicas.
 - Ofrece una amplia gama de herramientas para la autenticación de usuarios, la gestión de permisos y la protección contra amenazas de seguridad comunes como la inyección SQL y XSS.
 - Facilita la gestión de sesiones y el almacenamiento en caché de datos, lo que mejora el rendimiento de las aplicaciones web.
 - Deltanet Travel, Neighborhood Lender.
- **¿Cuándo utilizar?**
 - Se enfoca en la productividad y el desarrollo rápido para todo tipo de aplicaciones.

Slim (PHP)

Frameworks de backend



Slim Framework

- Lanzado en **2005**. Es un microframework ideal para pequeñas aplicaciones web y APIs. Slim enfoca su prioridad en la seguridad y velocidad de respuesta, a lo que contribuye su simplicidad y peso ligero. Slim ofrece terminar tu trabajo con rapidez y calidad.
- **Características**
 - Permite escribir rápidamente aplicaciones web y APIs.
 - Aporta un sistema de rutas.
 - Uso de middlewares para interceptar el flujo normal del proyecto y aportar funcionalidad.
 - Ofrece inyección de dependencias.
 - Slim es un framework de software libre y código abierto.
- **¿Cuándo utilizar?**
 - Aplicaciones pequeñas de manera rápida, flexible y sencilla.

Symfony (PHP)

Frameworks de backend

- Lanzado en **2005**. Symfony es un marco de desarrollo web de código abierto en PHP. Está diseñado para facilitar la construcción de aplicaciones web robustas y escalables utilizando PHP. Esta basado en el patrón Modelo Vista Controlador.
- **Características**
 - Se basa en un conjunto de componentes independientes que pueden ser utilizados de manera aislada en proyectos PHP, lo que facilita la incorporación de características específicas de Symfony en aplicaciones existentes.
 - Proporciona un sistema de enrutamiento flexible que permite definir rutas y controladores para manejar solicitudes HTTP.
 - Se integra bien con Doctrine, un ORM que facilita la interacción con bases de datos SQL.
 - Utiliza el motor de plantillas Twig que permite crear vistas. Twig ofrece características como herencia de plantillas y extensibilidad que simplifican la creación de vistas dinámicas.
 - Incluye una capa de seguridad robusta que facilita la gestión de autenticación, autorización y protección contra amenazas.
 - Utiliza el componente Dependency Injection para gestionar las dependencias de la aplicación.
 - Spotify, Trivago, Dailymotion, Doc Planner.
- **¿Cuándo utilizar?**
 - Se enfoca en la productividad y el desarrollo rápido para todo tipo de aplicaciones.

Express JS (Node JS)

Frameworks de backend



- Lanzado en **2010**. Se emplea, por lo regular, como marco de desarrollo minimalista y flexible para crear aplicaciones web y API utilizando Node.js, un entorno de tiempo de ejecución del lado del servidor. El objetivo principal del proyecto era facilitar la creación de aplicaciones web en comparación con un servidor web estándar como Apache HTTP Server.
- **Características**
 - Usa un sistema de enrutamiento que permite definir rutas y controladores para manejar solicitudes HTTP.
 - Usa un sistema de middleware que permite realizar acciones intermedias en el flujo de las solicitudes y respuestas HTTP. Esto es útil para realizar tareas como autenticación, registro de solicitudes, compresión de respuestas y mucho más.
 - Aunque Express en sí mismo no incluye un motor de plantillas, es utilizado en combinación con motores de plantillas como EJS, Pug o Handlebars para generar vistas dinámicas.
 - Puede servir archivos estáticos (como archivos HTML, CSS, imágenes, etc.) de forma directa.
 - Permite la gestión de sesiones y cookies, lo que facilita la creación de aplicaciones web que requieren autenticación y seguimiento de usuarios.
 - Es una elección popular para la creación de API RESTful debido a su capacidad para manejar rutas y controladores de manera eficiente.
 - Storify, Myspace, LearnBoost.
- **¿Cuándo utilizar?**
 - Su uso en los frameworks más populares como Angular y React lo hizo ideal para desarrollar el API para el backend.

Frameworks de frontend

Tipos de frameworks existentes

Tipos de framework para desarrollo web

Frameworks de frontend

- Frameworks de JavaScript

- React
- Vue.js
- Angular
- jQuery (jQuery UI)
- Ember
- Backbone.js

- Frameworks de CSS

- Bootstrap
- Tailwind CSS
- Bulma
- Foundation
- Semantic UI
- Materialize

React

Frameworks de frontend



- React, también conocido como React.js, es una biblioteca de JavaScript. Fue desarrollada por **Facebook** en **2013** y es una herramienta poderosa para crear interfaces de usuario interactivas y componentes reutilizables en aplicaciones web.
- **Características**
 - Divide la interfaz de usuario en **componentes reutilizables**. Cada componente puede representar una parte específica de la interfaz de usuario y contener su propia lógica y estado. Esto hace que el desarrollo sea modular.
 - Ofrece la **gestión de estado**, el **enrutamiento** o las **integraciones de API**.
 - Además, utiliza un **Virtual DOM** para optimizar el rendimiento de las actualizaciones de la interfaz de usuario, en lugar de actualizar directamente el DOM real cada vez que cambia el estado de la aplicación. React compara el Virtual DOM con el DOM actual y aplica solo las diferencias necesarias.
 - React usa **JSX**, lo que permite a los desarrolladores escribir fragmentos de HTML dentro de archivos JavaScript.
 - El enlace de datos es unidireccional que ayuda a los desarrolladores a anidar componentes.
 - Facebook, Netflix, Asana, Pinterest, Airbnb, Reddit, UberEats.
- **¿Cuándo utilizar?**
 - Debido a sus capacidades de DOM virtual, React es la opción óptima para desarrollar aplicaciones de página única (SPA). Muy bueno para interfaces con elementos repetitivos como tarjetas.

React

Frameworks de frontend



```
<div id="root"></div>
```

```
// Enviamos el componente al archivo public/index.html  
const root =  
ReactDOM.createRoot(document.getElementById('root'));  
root.render(<App />);
```

Vue.js

Frameworks de frontend



- Vue.js creada en **2014**, es un marco de trabajo que suele preferirse para aplicaciones frontend pequeñas que necesitan ejecutarse en un marco de JavaScript. Es un marco flexible al escribir código. Además, el marco es conocido por su tamaño compatible y tiempos de carga más rápidos.
- **Características**
 - Vue.js aprovecha **Virtual DOM** para cambios de estado y actualizaciones.
 - Utiliza un **enlace de datos bidireccional**, es decir, que los cambios en los datos de la aplicación se reflejan de manera automática en la interfaz de usuario y viceversa, sin necesidad de manipular el DOM directamente.
 - Es conocido por ser un **framework progresivo**, lo que significa que puede usarse de manera incremental en los proyectos.
 - Vue.js también utiliza **JSX**. El uso de JSX permite a Vue.js importar y administrar componentes personalizados fácilmente.
 - Vue.js tiene muchas funciones con **transiciones y animaciones CSS**, ya que ofrece numerosas formas de aplicar efectos de transición.
 - Alibaba, 9gag, Xiaomi.
- **¿Cuándo utilizar?**
 - Ayuda a construir aplicaciones de página única (SPA) o lanzar pequeños proyectos. Es relativamente liviano y fácil de integrar.

Vue.js

Frameworks de frontend



```
<div id="app">
  {{ message }}
</div>
```

```
new Vue({
  el: '#app',
  data: {
    message: 'Hello Vue.js!'
  }
})
```

Angular

Frameworks de frontend



- Angular es un marco de aplicación web de código abierto basado en TypeScript desarrollado por **Google** en **2016**. La versión 2 de este marco introdujo características principales como *dependency injection*, compilaciones asincrónicas, RxJS (Reactive Extensions for JS).
- **Características**
 - Su enfoque está en la arquitectura de **componentes**. Divide la interfaz de usuario en componentes reutilizables, lo que facilita la organización y el mantenimiento del código. Cada unidad se encarga de su propia lógica y vista, lo que promueve una estructura modular y limpia.
 - Angular es un marco **MVC** completo. Ayuda a crear una aplicación estructurada.
 - Angular también utiliza **TypeScript**, un superset de JavaScript que agrega tipado estático.
 - El marco ofrece **flujo de datos bidireccional**. Esta característica permite a Angular conectar DOM a los datos del modelo a través del controlador. Al utilizar el flujo de datos bidireccional, puede escuchar eventos y actualizar los datos simultáneamente entre los componentes.
 - Si bien este entorno puede tener una **curva de aprendizaje más pronunciada** en comparación con algunos otros, su potencial y sus características avanzadas lo convierten en una elección sólida para aplicaciones web complejas y empresariales.
 - Forbes, LEGO, Autodesk, BMW, Google.
- **¿Cuándo utilizar?**
 - Angular + Node Express server, al ser un marco completo es adecuado para desarrollar aplicaciones empresariales a gran escala. Además, dado que también utiliza la arquitectura MVC, permite integraciones perfectas de funciones avanzadas.

Angular

Frameworks de frontend



```
<app-root></app-root>
```

```
@Component({  
  selector: 'app-root',  
  template: `<h1>{{name}}</h1>`,  
})
```

```
export class AppComponent { name = 'hello-world'; }
```

- Antes del nacimiento de las bibliotecas/marcos modernos de JavaScript, jQuery dominaba el mercado de TI. Lanzada en **2006** es una de las bibliotecas de JavaScript de código abierto más antiguas, liviana y rica en funciones.
- **Características**
 - jQuery es conocido por sus capacidades de **manejo de eventos**. Incluso las actividades del usuario, como hacer clic con el mouse o pulsar el teclado, se reducen a pequeños fragmentos de código para facilitar la administración.
 - La sintaxis de jQuery es bastante **similar a la de CSS**. Como resultado, es bastante fácil para los principiantes adquirir experiencia práctica con el marco.
 - Incluye **jQuery Mobile** para crear aplicaciones móviles escalables.
 - Es un marco **fácil de usar** con una API minimalista que admite múltiples navegadores.
 - Se considera uno de los mejores marcos de JavaScript frontend. Se calcula que lo utiliza el **95% de todos los sitios web** que utilizan JavaScript.
 - Twitter, Microsoft, Uber, Pandora, SurveyMonkey.
- **¿Cuándo utilizar?**
 - Con soporte para varios navegadores y usando en conjunto con componentes de terceros, jQuery sigue siendo ideal para ofrecer aplicaciones JavaScript basadas en web.

```
<div id="container">This will change to Hello World!</div>
```

```
<script>  
  $(document).ready(function() {  
    $("#container").text("Hello World");  
  });  
</script>
```

Ember

Frameworks de frontend



- Lanzado en **2011**, Ember es un marco de JavaScript de código abierto que admite la arquitectura MVC y MVVM. Ember se basa en un motor de renderizado Glimmer que es conocido por ser uno de los motores de renderizado más rápidos del mercado. El enfoque principal de Ember es la creación de aplicaciones basadas en web. Pero, con el tiempo, Ember ha evolucionado y ahora ofrece soporte completo para crear aplicaciones móviles.
- **Características**
 - Ember ofrece **enlace de datos bidireccional** que sincroniza la vista y el modelo en tiempo real.
 - Ember aprovecha **fastboot.js + node.js** para mejorar el rendimiento de interfaces de usuario complejas a través de su representación de DOM en el lado del servidor.
 - Admite **JavaScript** y **TypeScript**.
 - Apple Music, Square, LinkedIn, Netflix, Twitch.
- **¿Cuándo utilizar?**
 - Idealmente adecuado para aplicaciones simples de una sola página (SPA).

Ember

Frameworks de frontend



```
<script type="text/x-handlebars" data-template-name="index">  
  <div>  
    <h1>{{title}}</h1>  
</script>
```

```
import Ember from 'ember';  
export default Ember.Controller.extend({  
  title: "Hello World"  
});
```

Backbone.js

Frameworks de frontend



- Lanzado en **2010**, Backbone.js es una biblioteca JavaScript de código abierto. Se caracteriza por ser liviano. Sin embargo, el marco depende en gran medida de underscore.js y jQuery para su soporte completo de biblioteca.
- **Características**
 - Backbone.js incluye una de las características principales para **separar la lógica** de entrada del usuario y del negocio. Esto ayuda a los desarrolladores a estructurar correctamente la aplicación.
 - Backbone.js anima a los desarrolladores a convertir **datos en modelos**, manipulaciones DOM en **vistas** y vincularlos a ambos mediante **eventos**.
 - El marco proporciona API enriquecidas de **funciones enumerables** para aplicaciones web del lado del cliente.
 - En backbone.js, cada vez que cambia el **modelo de arquitectura**, actualiza automáticamente el HTML de su aplicación.
 - Trello, Tumblr, Uber, Pinterest y Reddit.
- **¿Cuándo utilizar?**
 - Idealmente adecuado para aplicaciones simples de una sola página (SPA).

Backbone.js

Frameworks de frontend



```
<body></body>

<script>
  $( function(){
    (function () {
      var View = Backbone.View.extend({
        "el": "body",
        "template": _.template("<p>Hello World!</p>"),

        "initialize": function () {
          this.render();
        },
        "render": function () {
          this.$el.html(this.template());
        }
      });

      new View();
    })()
  } );
</script>
```

Tipos de framework para desarrollo web

Frameworks de frontend

- Frameworks de JavaScript

- React
- Vue.js
- Angular
- jQuery (jQuery UI)
- Ember
- Backbone.js

- Frameworks de CSS

- Bootstrap
- Tailwind CSS
- Bulma
- Foundation
- Semantic UI
- Materialize

Bootstrap

Frameworks de frontend



- Bootstrap comenzó en **2011** como un proyecto paralelo creado y compartido por desarrolladores de Twitter. Ahora es el marco CSS más utilizado para el diseño web responsivo y orientado a dispositivos móviles. Millones de personas utilizan Bootstrap para crear diseños web limpios, consistentes y familiares.
- **Características**
 - Sistema de red responsivo.
 - Componentes de interfaz de usuario prediseñados.
 - Temas personalizables y extensibles.
 - Amplia documentación.
 - Twitter, Microsoft, Spotify, Udacity.
- **¿Cuándo utilizar?**
 - Para todo tipo de proyectos. Es simple, limpio y fácil de usar.

Tailwind CSS

Frameworks de frontend



- Creado en 2017 donde su creador escribió una especie de manifiesto sobre el pensamiento detrás de Tailwind . Y esencialmente, la idea es que CSS no debería ser descriptivo y semántico (por ejemplo, la clase ".header"), sino que debería ser funcional (por ejemplo, ".center-flex-3"). Lo llama un marco CSS de utilidad.
- **Características**
 - Clases de utilidad para un estilo sencillo.
 - Capacidades de diseño responsivo.
 - Configuración personalizable.
 - Enfoque amigable con los componentes.
 - OpenAI (ChatGPT), Shopify, Wealthfront y Loom.
- **¿Cuándo utilizar?**
 - Para todo tipo de proyectos. No existen componentes prediseñados de la misma manera que, por ejemplo, Bootstrap. En su lugar, puede combinar y superponer clases para colocar sus elementos HTML en una cantidad casi infinita de diseños y cuadrículas CSS.

Bulma

Frameworks de frontend



- Lanzado en **2016**. Bulma es un marco CSS ligero basado en Flexbox. La sintaxis de este marco es sencilla, lo que significa que se basa en gran medida en clases de utilidad descriptivas o modificadores como ".button" y ".is-large".
- **Características**
 - Sistema de cuadrícula basado en Flexbox.
 - Arquitectura modular.
 - Con tecnología Sass (Syntactically Awesome Style Sheets) para una fácil personalización.
 - Código y diseño minimalistas.
 - Ejemplos: CSS Ninja y Signal.
- **¿Cuándo utilizar?**
 - Es simple y fácil de entender, lo que lo hace ideal para principiantes o personas que sólo desean una solución rápida. Para proyectos medianos pues carece de la sofisticación o extensibilidad de opciones más sólidas.

Foundation

Frameworks de frontend



- Lanzado en **2011**. Se encuentra en el extremo opuesto del espectro de Bulma. Foundation no tiene reparos en ser avanzado y, como tal, bastante complejo en comparación con algunas de las otras opciones. Es un marco de interfaz de usuario responsivo diseñado para el desarrollo móvil primero y utilizado tanto para sitios como para correos electrónicos.
- **Características**
 - Se denomina a si mismo como "El framework frontend responsivo más avanzado del mundo".
 - Sistema de grid responsivo.
 - Conjunto completo de componentes de interfaz de usuario.
 - Variables Sass personalizables.
 - Integración con bibliotecas y herramientas de frontend populares.
 - Twitter, Microsoft, Spotify, Udacity.
- **¿Cuándo utilizar?**
 - Para proyectos grandes y que requieran mucha personalización. La curva de aprendizaje es un poco pronunciada.

Semantic UI

Frameworks de frontend



- Lanzado en **2013**. Es un framework para crear el diseño de interfaces de manera responsiva utilizando HTML/CSS legible, al igual que otros frameworks, se basa en componentes tales como grids, botones, formularios, headers, menús, entre otros.
- En esencia, este marco está diseñado para ayudar a construir y escalar interfaces familiares para sitios web y aplicaciones web.
- **Características**
 - Nombres de clases intuitivos y legibles por humanos.
 - Amplia gama de componentes y diseños de interfaz de usuario.
 - Temas y estilos personalizables.
 - Integración con bibliotecas y marcos de JavaScript populares como Angular.
 - Accenture y Snapchat.
- **¿Cuándo utilizar?**
 - Incluye una biblioteca de temas con más de 3000 variables personalizables. Puede resultar demasiado voluminoso para proyectos simples.

Materialize

Frameworks de frontend



- Lanzado en **2014**. Materialize es un framework CSS responsive basado en Material Design de **Google**. Enfatiza el diseño visual audaz y la animación (movimiento) centrada en UX.
- **Características**
 - Componentes y estilos inspirados en el diseño de materiales.
 - Sistema de red responsivo.
 - Personalización impulsada por Sass.
 - Complementos de JavaScript integrados.
 - Twitter, Microsoft, Spotify, Udacity.
- **¿Cuándo utilizar?**
 - Proyectos pequeños pues no es tan robusto ni extensible como otros marcos y depende de JavaScript para movimientos específicos.

Marcos de trabajo (frameworks) para el desarrollo de aplicaciones web



FRAMEWORKS

Spring

★ Star 25K 🍴 Fork 16K

Dropwizard

★ Star 6K 🍴 Fork 2K

Blade

★ Star 4K 🍴 Fork +926

TESTING

Mockito

★ Star 7K 🍴 Fork 1K

Junit

★ Star 7K 🍴 Fork 2K

TestNG

★ Star 1K 🍴 Fork +714



FRAMEWORKS

Laravel

★ Star 47K 🍴 Fork 14K

Symfony

★ Star 19K 🍴 Fork 6K

CodeIgniter

★ Star 16K 🍴 Fork 7K

CakePHP

★ Star 7K 🍴 Fork 3K

Phalcon

★ Star 9K 🍴 Fork 1K

TESTING

PHPUnit

★ Star 12K 🍴 Fork 1K

Codeception

★ Star 3K 🍴 Fork 1K

PHPSpec

★ Star 1K 🍴 Fork +254



FRAMEWORKS

Django

★ Star 38K 🍴 Fork 16K

Pyramid

★ Star 3K 🍴 Fork +835

Web2py

★ Star 1K 🍴 Fork +801

MICROFRAMEWORKS

Flask

★ Star 40K 🍴 Fork 11K

Bottle

★ Star 3K 🍴 Fork +835

FRAMEWORKS ASINCRONOS

Tornado

★ Star 16K 🍴 Fork 4K

Sanic

★ Star 10K 🍴 Fork 1K

TESTING

PyTest

★ Star 3K 🍴 Fork 866

Nose

★ Star 1K 🍴 Fork +346



FRAMEWORKS

Express.js

★ Star 41K 🍴 Fork 7K

Koa2

★ Star 24K 🍴 Fork 2K

Sails.js

★ Star 19K 🍴 Fork 1K

NestJS

★ Star 10K 🍴 Fork +699

TESTING

Jest

★ Star 22K 🍴 Fork 2K

Mocha

★ Star 16K 🍴 Fork 2K

Jasmine

★ Star 14K 🍴 Fork 2K

Chai

★ Star 5K 🍴 Fork +539



FRAMEWORKS

Ruby on Rails

★ Star 41K 🍴 Fork 16K

Sinatra

★ Star 10K 🍴 Fork 1K

Padrino

★ Star 3K 🍴 Fork +494

TESTING

Capybara

★ Star 8K 🍴 Fork 1K

RSpec

★ Star 2K 🍴 Fork +193



FRAMEWORKS

Echo

★ Star 12K 🍴 Fork 1K

Beego

★ Star 18K 🍴 Fork 6K

Iris

★ Star 13K 🍴 Fork 1K

Revel

★ Star 10K 🍴 Fork 1K

TESTING

Paquete "testing" de Golang

¿Cómo elegir el mejor framework?

Implementación de frameworks para el desarrollo Web

- Es difícil realizar un seguimiento de todas las diferencias entre un marco. Lo que es más desafiante es decidir cuál elegir para su proyecto. Una decisión equivocada puede suponer una pésima apuesta para su tiempo y presupuesto del proyecto. Desafortunadamente, no existe una manera fácil de identificar si un marco en particular es el mejor para usted.
- Elegir el mejor marco depende de los requisitos de su negocio, y los siguientes cuatro factores pueden ayudarlo en la toma de decisiones:
 - Competencia del equipo.
 - Compatibilidad con el servidor.
 - Complejidad.
 - Tamaño y rendimiento.

Competencia del equipo

¿Cómo elegir el mejor framework?

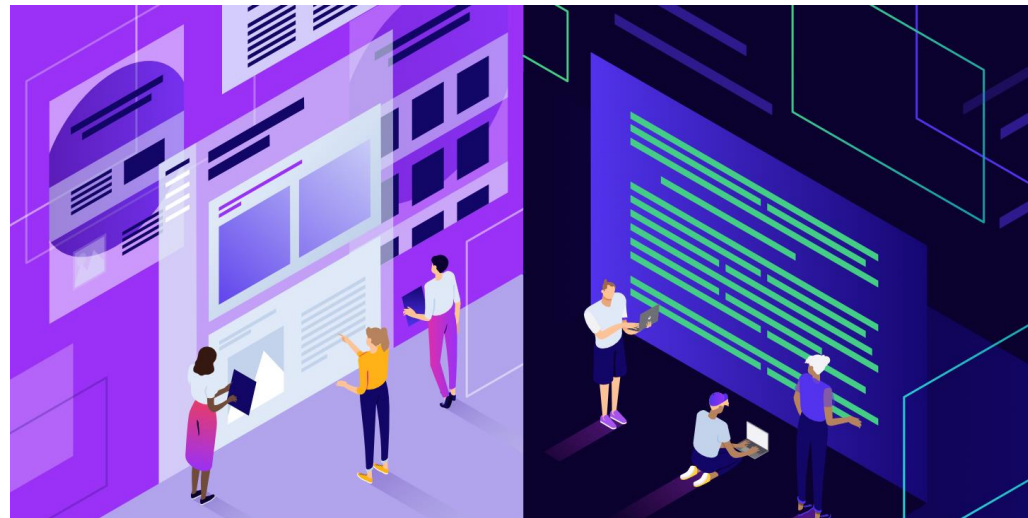
- Un marco podría ser el más destacado que existe. Pero si su equipo actual no se siente cómodo usándolo, los desarrolladores no podrán utilizar este marco en todo su potencial.
- Entonces, elija un marco con el que su equipo tenga más experiencia o puede contratar recursos remotos para aprovechar al máximo los marcos populares del mercado.



Compatibilidad con el servidor

¿Cómo elegir el mejor framework?

- Un marco de frontend ayuda a codificar solo la mitad de la aplicación, y también es necesario cuidar el backend. Es necesario que haya una conectividad armoniosa entre el frontend y el backend para una aplicación completamente operativa.
- Si ya ha desarrollado su backend, es aconsejable elegir un marco de frontend que coincida con su backend. De lo contrario, se verá abrumado por desafíos, problemas de compatibilidad, problemas de integración, etc.



Complejidad

¿Cómo elegir el mejor framework?

- Si está desarrollando una aplicación web compleja con cientos de subpáginas y funcionalidades, elija un marco basado en la arquitectura MVC. Hacerlo le permitirá separar sus funcionalidades en partes que podrían desarrollarse simultáneamente. Esto reducirá significativamente su tiempo de desarrollo.
- Elegir un marco liviano para desarrollar aplicaciones complejas puede resultar catastrófico. Un marco más ligero no puede soportar la complejidad y sufrirá multitud de problemas en el futuro.



Tamaño y rendimiento

¿Cómo elegir el mejor framework?

- Lo ideal es que un marco no tenga muchas funciones todo el tiempo. Algunos marcos pueden parecer súper avanzados, pero serán difíciles de implementar. Si se está concentrando en construir un sistema simple con funcionalidades limitadas, concéntrese en obtener el máximo rendimiento.
- Como regla general, intente siempre equilibrar el rendimiento y el tamaño para obtener la mejor eficiencia de un marco.



Seleccionar herramientas de trabajo principales

Consideraciones para el desarrollo web

- Herramientas para desarrolladores (*Setting up your toolbox*)
 - **Editores.** Un editor es donde se escribe el código y, a veces, donde se ejecuta. Características: Depuración, Resaltado de sintaxis, Extensiones e integraciones, Personalización.
VS Code, Sublime, Eclipse, Atom, Notepad++.
 - **IDE.** Un *entorno integrado de desarrollo* es un conjunto único de herramientas y características que un desarrollador puede usar para escribir software.
PyCharm, VS Studio, WebStorm, Netbeans, PhpStorm, RubyMine, PyDev, IntelliJ Idea, Xcode
- Tecnologías del explorador web
 - Chrome, Safari, Edge, Firefox
 - Herramientas para desarrolladores en navegadores
- Línea de comandos
 - PowerShell, Terminal, Bash, **Git**

Seleccionar herramientas de trabajo complementarias

Consideraciones para el desarrollo web

- Administradores de paquetes
 - Rubygems (ruby), yarn (Facebook, react), npm (node), bower (js), nuget (.net), maven (java), composer (php), pip (python)
- Linting (análisis de código)
 - ESLint (js)
- Formateo
 - Prettier
- Documentación
 - <https://developer.mozilla.org/>
 - <https://www.w3schools.com/>
 - <https://caniuse.com/>



Abrir los enlaces



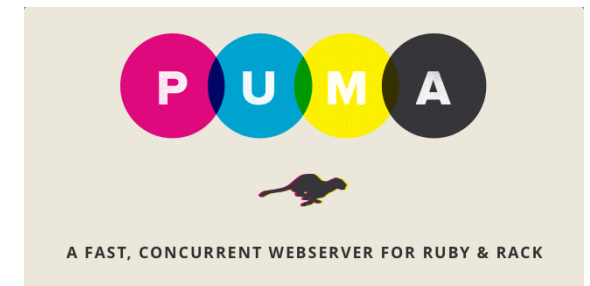
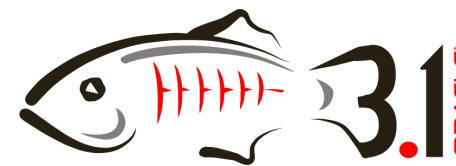
Seleccionar servidores web

Consideraciones para el desarrollo web

- Servidores web

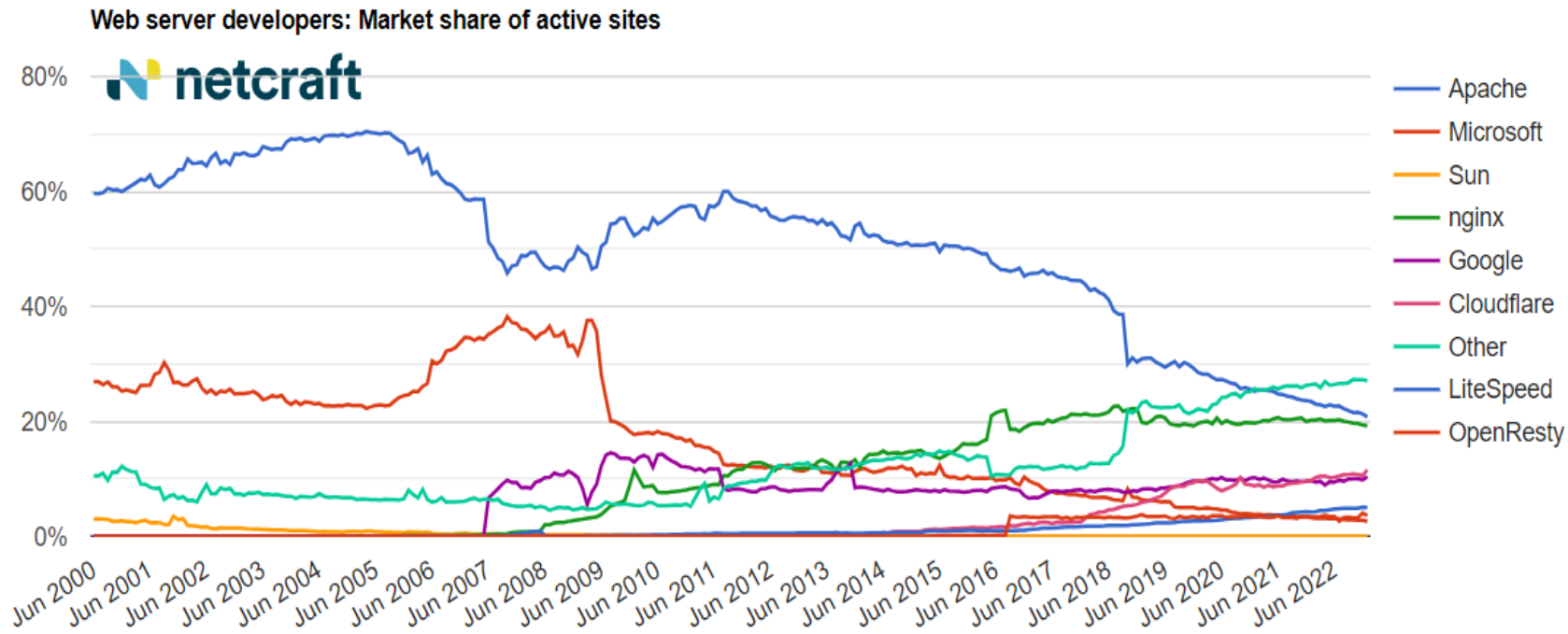
- Apache HTTP
- Nginx, OpenResty
- LiteSpeed
- IIS, Kestrel
- Lighttpd
- Sun Java System Web Server, Glashfish, Tomcat
- NodeJS, Express
- Tornado, Daphne, Uvicorn (python)
- Puma, unicorn, thin (ruby)

Un servidor web o servidor HTTP es un programa informático que procesa una aplicación del lado del servidor, realizando conexiones bidireccionales o unidireccionales y síncronas o asíncronas con el cliente y generando una respuesta del lado del cliente.



Servidores web más populares

Servidores web



Nginx	34.2%
Apache	31.1%
Cloudflare Server	21.0%
LiteSpeed	12.5%
Microsoft-IIS	5.3%
Node.js	2.9%
Google Servers	0.9%
Envoy	0.4%
Caddy	0.2%
IdeaWebServer	0.1%
Tengine	0.1%
Cowboy	0.1%
Kestrel	0.1%
ArvanNginx	0.1%
Tomcat	0.1%

W3Techs.com, 21 September 2023

Percentages of websites using various web servers
Note: a website may use more than one web server

<https://www.netcraft.com/blog/january-2023-web-server-survey/>

https://w3techs.com/technologies/overview/web_server

Seleccionar bases de datos

Consideraciones para el desarrollo web

- Bases de datos
 - PostgreSQL
 - MySQL
 - MariaDB
 - Oracle
 - MongoDB (noSQL)
 - Microsoft SQL Server
 - Redis (memoria)
 - Apache Cassandra (distribuida)
 - Amazon DynamoDB (noSQL)

Un sistema gestor de base de datos o SGBD es un software que permite administrar una base de datos.



DynamoDB



Seleccionar servicios de hospedaje web

Consideraciones para el desarrollo web

- Hosting
 - **Amazon Web Services (AWS)**
 - **Google Cloud Platform**
 - **Azure**
 - Hostinger
 - SiteGround
 - HostGator
 - WebEmpresa
 - DreamHost
 - Go Daddy
 - Digital Ocean

El alojamiento web u hospedaje web es el servicio que provee a los usuarios de Internet un espacio de almacenamiento en línea, también conocido como hosting, que permite publicar todo el contenido relacionado con una aplicación web.

Crear un App en Azure gratis



Conclusiones

Unidad II. Frameworks de desarrollo web

- El uso de frameworks es una de las técnicas más usadas por los programadores para el desarrollo web. No es para menos, pues este tipo de software brinda un sinnúmero de facilidades para centrarse en la lógica del funcionamiento del sitio, en lugar de gastar el tiempo en la escritura de códigos que se pueden reutilizar.
- Se trata de soluciones versátiles y funcionales que han facilitado la tarea de la programación. Trabajan en módulos que ayudan en la configuración de funciones y bloques dentro del desarrollo.
- La elección del framework dependerá de varios factores como la familiaridad del desarrollador, los requisitos del proyecto, la escalabilidad y la comunidad de soporte.

Estructura de las aplicaciones web

Unidad 4. Estructura de las aplicaciones web

Saberes teóricos

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- **Unidad 4. Desarrollo de Aplicaciones web dinámicas**
 - Taxonomía de las aplicaciones web
 - Marcos de trabajo (frameworks) para el desarrollo de aplicaciones web
 - Aplicaciones web para dispositivos móviles
 - Servidores web
 - Estructura de las aplicaciones web
 - Modelos de capas
 - Separación de estructura, presentación y comportamiento
 - Entrega de contenido multimedia en la web

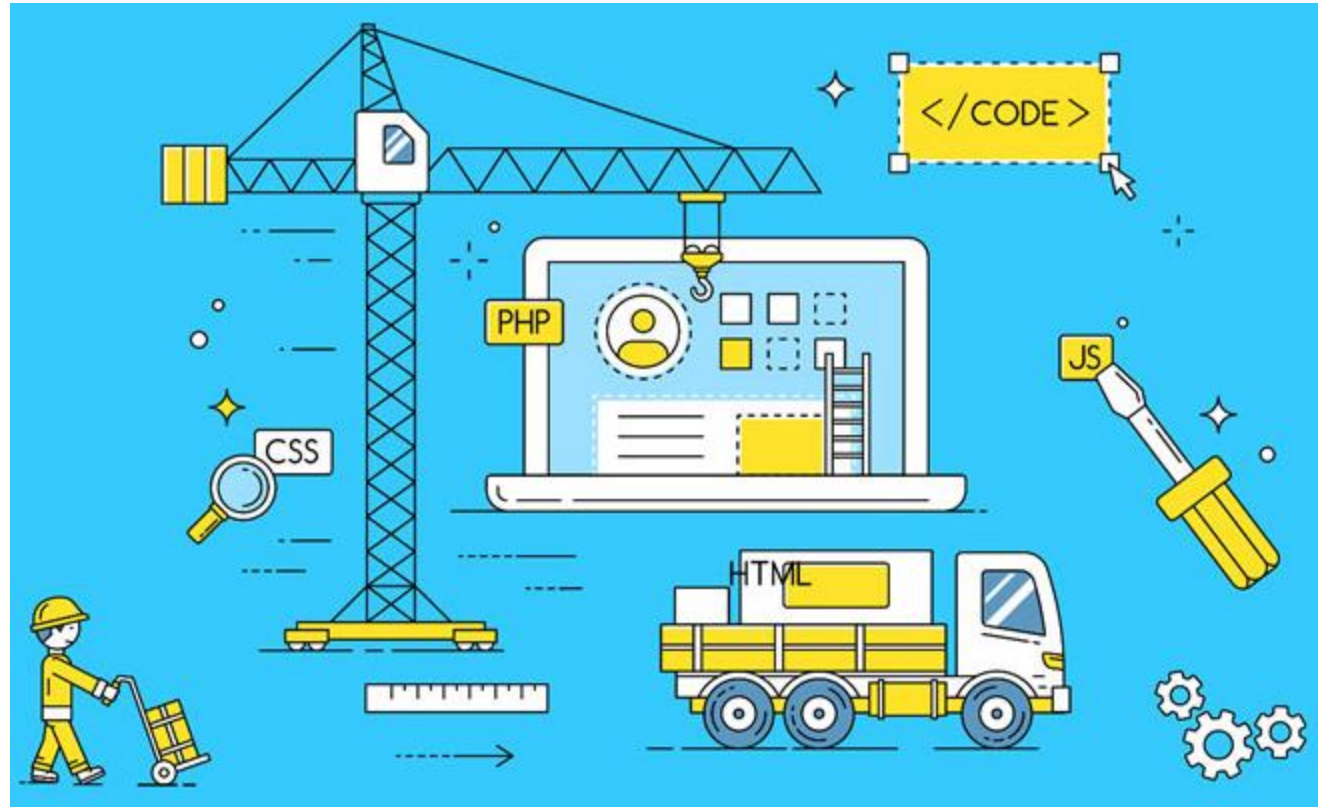
Introducción

Introducción Estructura de las aplicaciones web

- La estructura de las aplicaciones Web es otorgada por la arquitectura del software, definiendo cómo se organizan, interactúan y despliegan los componentes (frontend, backend, BD) para garantizar **escalabilidad, mantenibilidad** y rendimiento.
- La arquitectura es el **plano estratégico** (conceptual) que define el comportamiento global, mientras que la estructura es la **implementación** física de ese plan.

¿Qué es una arquitectura de software?

Estructura de las aplicaciones web



La **arquitectura de software** se define como el conjunto de componentes de un sistema, las interfaces de comunicación de los mismos, y la manera como estos componentes se comunican entre ellos usando estas interfaces.

¿Qué es una arquitectura de software?

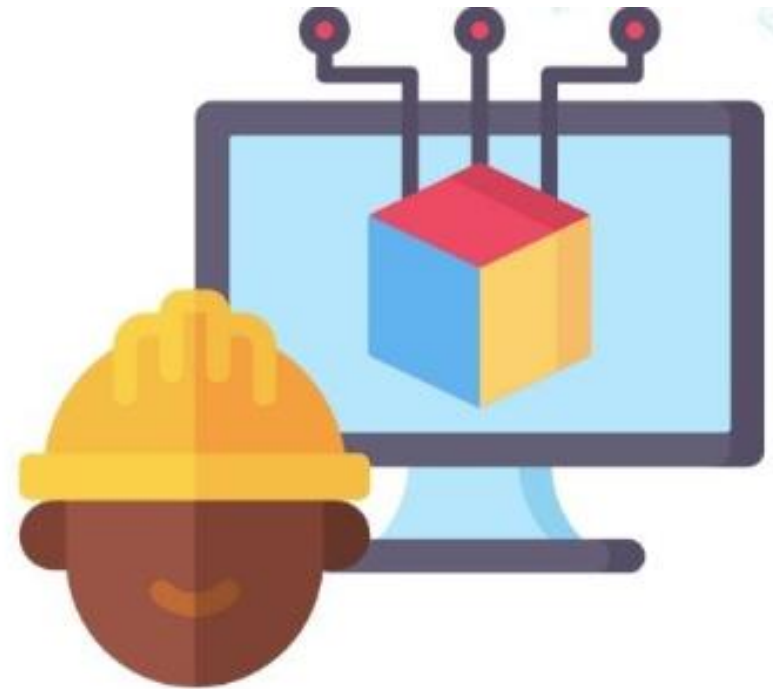
Arquitectura de software

- Es una representación de alto nivel que define cómo los **componentes** del software interactúan entre sí, cómo se **organizan** y cómo cumplen con los **requisitos** funcionales y no funcionales del sistema.
- La arquitectura de software proporciona una visión global del sistema, lo que permite a los desarrolladores y arquitectos comprender su estructura y tomar decisiones informadas durante el proceso de desarrollo.
- Una buena arquitectura de software es fundamental para el éxito de un proyecto, ya que afecta a la **calidad**, el **rendimiento** y la **escalabilidad** del software.
- Además, facilita la **colaboración** entre los miembros del equipo de desarrollo y brinda una **visión** clara de cómo se estructura el sistema, lo que ayuda a minimizar problemas y errores a medida que avanza el desarrollo del software.

¿Qué es un arquitecto de software?

Arquitectura de software

Es el encargado de desarrollar la propuesta técnica para **crear plataformas digitales**. Desde la definición de la estructura hasta los estándares de código que se debe implementar.



¿Qué hace un arquitecto de software?

Arquitectura de software

- Identificar las necesidades negocio y viabilidad de soluciones.
- Diseñar la **estructura del sistema**.
- Elegir las tecnologías para cada componente.
- Liderar equipos de trabajo.
- Seguimiento a la arquitectura y soluciones.
- Coordinar la documentación.
- Establecer estándares de desarrollo.
- Escuchar y argumentar la toma de decisiones con los stakeholders.



Elementos de la arquitectura de software

Arquitectura de software

La arquitectura de software se conforma mediante la combinación de varios elementos y conceptos. Estos elementos fundamentales se combinan para definir la estructura y el diseño del sistema de software:

- Componentes.
- Conexiones.
- Estilo arquitectónico.
- Patrones de diseño.
- Requisitos no funcionales.
- Tecnologías y herramientas.
- Documentación.
- Consideraciones de evolución y mantenimiento.

Componentes

Elementos de la arquitectura de software

Los componentes son los módulos, servicios o partes del software que realizan tareas específicas dentro del sistema. Estos pueden incluir componentes de:

- Componentes de **interfaz de usuario**. HTML, CSS, y JavaScript.
- **Lógica de negocio**. Procesando las entradas del usuario, aplicando reglas de negocio.
- **Acceso a bases de datos**. Gestionan la lectura, escritura, actualización y borrado de datos
- **Middleware y servicios web**. Servidores web, servidores de aplicaciones.
- **Hardware y software**. Como servidores, sistemas operativos, contenedores, y plataformas en la nube.
- **Seguridad**. Mecanismos de autenticación, autorización, cifrado, y protección contra ataques comunes
- **Gestión y monitoreo**. Rendimiento de la aplicación, diagnosticar problemas, y gestionar la configuración y despliegue.
- **Redes y comunicación**. Protocolos de red, balanceadores de carga, y sistemas de nombres de dominio (DNS).

Conexiones

Arquitectura de software

Conexiones

Elementos de la arquitectura de software

Las conexiones representan cómo los componentes se comunican y colaboran entre sí. Esto incluye la definición de interfaces, protocolos de comunicación y flujos de datos.

- **Conexiones de red.** HTTP/HTTPS, WebSocket, TCP/IP.
- **Apis** (interfaces de programación de aplicaciones). REST, SOAP, GraphQL
- **Buses de mensajería** y sistemas de colas.
- **Conexiones a bases de datos.** JDBC, ODBC, ORM.
- **Conexiones de servicios externos.** Servicios de pago, redes sociales, servicios de geolocalización.
- **Middleware** y servicios de intermediación.
- **Conexiones de seguridad.** TLS/SSL, autenticación y autorización.
- **Interfaces gráficas de usuario** (GUI): Conexión entre la interfaz y la lógica del negocio mediante eventos, entradas de datos y comandos.

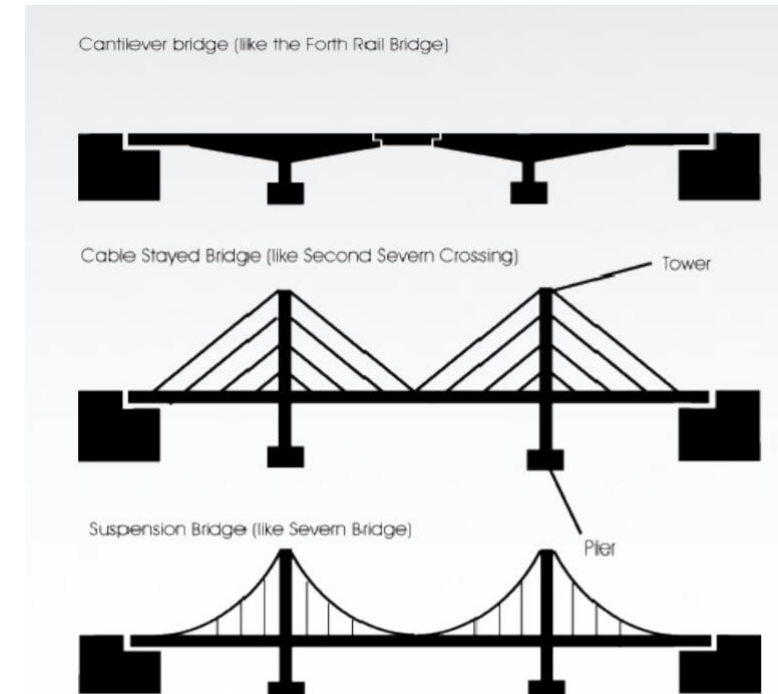
Estilo arquitectónico

Arquitectura de software

Estilo arquitectónico

Elementos de la arquitectura de software

- Los ingenieros civiles, cuando tienen que construir un **punto**, utilizan una arquitectura.
- Un puente sirve para comunicar dos extremos de tierra separados. El resultado siempre será un puente.
- Pero generalmente seleccionan un tipo de puente determinado que se adapte a las necesidades del contexto y del problema a resolver.
- Seleccionan un **estilo de arquitectura**.



Estilo arquitectónico

Elementos de la arquitectura de software

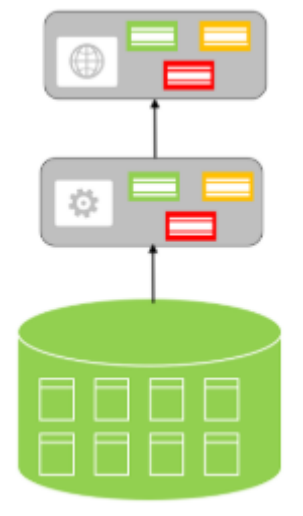
Los estilos arquitectónicos, también conocidos como **patrones arquitectónicos**, son un conjunto de **principios** que **guían** la organización, las decisiones de diseño y el patrón estructural de los sistemas de software.

Estos estilos definen la **estructura** de **alto nivel** del sistema, incluyendo la disposición de sus componentes y las interacciones entre ellos. Pueden **combinarse** para crear un estilo híbrido.

- Arquitectura cliente-servidor.
- Arquitectura por capas.
- Arquitectura monolítica.
- Arquitectura orientada a servicios (SOA).
- Arquitectura de microservicios.
- Arquitectura basada en componentes.
- Arquitectura en tubo y filtro (Pipe and Filter).
- Arquitectura Peer-to-Peer (P2P).
- Arquitectura de mensajes.
- Arquitectura basada en nube (Cloud-Native).

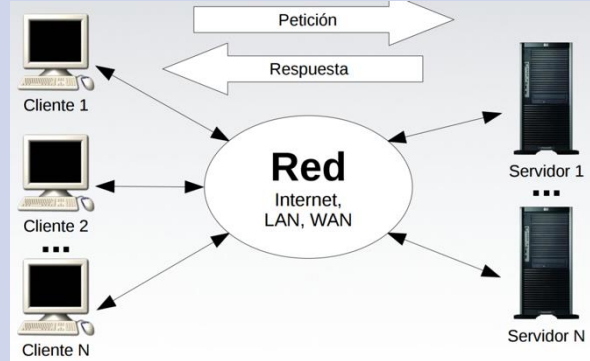
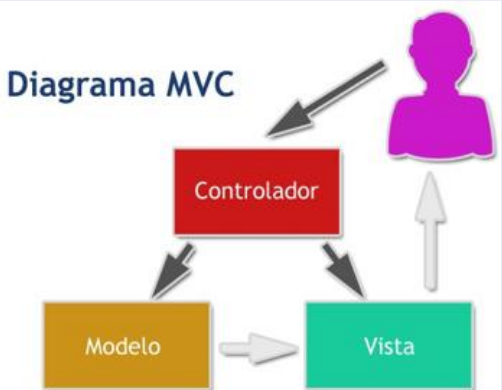
Estilos de Arquitectura

Unidad 4. Estructura de las aplicaciones web

Estilo de arquitectura	Descripción	Diagrama
Monolítica	Describe una aplicación en la que toda la funcionalidad del sistema (ej. acceso a datos, interfaz de usuario, lógica, etcétera) está implementada y mezclada en una sola capa.	
Capas o 3 niveles	Es una arquitectura tradicional para aplicaciones empresariales. Las dependencias se administran mediante la división de la aplicación en capas que realizan funciones lógicas como presentaciones, lógica de negocios y acceso a datos. Cada nivel se ejecuta en su propia infraestructura, cada nivel puede ser desarrollado simultáneamente por un equipo de desarrolladores distinto y se puede actualizar o escalar según sea necesario sin que afecte a los demás niveles.	

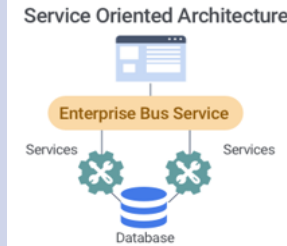
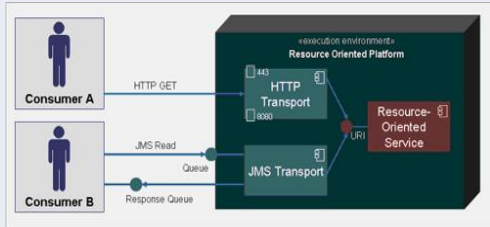
Estilos de Arquitectura

Unidad 4. Estructura de las aplicaciones web

Estilo de arquitectura	Descripción	Diagrama
Cliente-servidor	Consta de dos partes: un servidor que da servicio a múltiples clientes. En esta arquitectura se pueden separar las capas del servidor en varias máquinas físicas.	
Modelo-Vista-Controlador	Separa los datos de una aplicación (Modelo), la interfaz de usuario (Vista), y la lógica de control (Controlador) en tres componentes distintos.	

Estilos de Arquitectura

Unidad 4. Estructura de las aplicaciones web

Estilo de arquitectura	Descripción	Diagrama
Orientada al Servicio (SOA)	Se forma la estructura una aplicación descomponiéndola en varios servicios (normalmente como servicios HTTP) que se comunican por medio de un servicio de bus de mensajes.	
Orientada al Recurso (ROA)	Servicios Web implementados a través del protocolo HTTP o protocolos similares, con la restricción de establecer la interfaz a un conjunto conocido de operaciones estándar (GET, POST, PUT, DELETE). Se centra en interactuar con recursos con estado, más que con mensajes y operaciones. REST (Representational State Transfer).	
Microservicios	Consta de una colección de servicios autónomos y pequeños. Los servicios son independientes entre sí y cada uno debe implementar una funcionalidad de negocio individual. Los servicios están acoplados de forma flexible y se comunican a través de contratos de API.	

Requisitos no funcionales

Arquitectura de software

Requisitos no funcionales

Elementos de la arquitectura de software

Los requisitos no funcionales son los atributos de calidad que debe cumplir la arquitectura del software, como:

- **Rendimiento:** Se refiere a la respuesta del sistema ante determinadas cargas de trabajo. Incluye tiempos de respuesta, tasa de transacciones, y la capacidad de procesamiento.
- **Disponibilidad:** La capacidad del sistema de estar operativo y accesible cuando se requiere para su uso. Esto puede incluir mecanismos para recuperación ante fallos y redundancia.
- **Escalabilidad:** La habilidad del sistema para adaptarse a incrementos de carga, ya sea mediante la adición de recursos hardware o mediante la modificación de la aplicación. Incluye escalabilidad horizontal (añadir más máquinas) y vertical (añadir más recursos a una máquina).
- **Seguridad:** Comprende la protección de la información y los procesos del sistema contra robos, pérdidas o alteraciones, así como la protección contra el acceso o uso no autorizado. Incluye autenticación, autorización, cifrado, y auditoría.
- **Mantenibilidad:** La facilidad con la que el sistema puede ser modificado para corregir defectos, mejorar funcionalidades, o adaptarse a cambios en el entorno. Esto incluye la legibilidad del código, la modularidad y la reusabilidad.

Requisitos no funcionales

Elementos de la arquitectura de software

- **Portabilidad:** La facilidad con la que el software puede ser trasladado de un entorno de hardware o software a otro. Esto incluye la adaptabilidad del software a diferentes sistemas operativos, plataformas de hardware y navegadores.
- **Usabilidad:** Se refiere a la facilidad con la que los usuarios pueden aprender y utilizar el sistema. Incluye la interfaz de usuario, la documentación y el soporte técnico.
- **Confiabilidad:** La capacidad del sistema para funcionar bajo condiciones específicas durante un periodo determinado. Esto implica la reducción de fallos y la capacidad del sistema para recuperarse de estos.
- **Interoperabilidad:** La capacidad del sistema para trabajar con otros sistemas, intercambiar información y utilizar la información que ha sido intercambiada.
- **Compliance (Cumplimiento):** La adhesión a normativas, estándares y regulaciones relevantes en la industria y el ámbito legal. Esto puede incluir cumplimiento con GDPR para la protección de datos, normas ISO para calidad de software, y más.
- **Sostenibilidad:** La capacidad del sistema para ser operado y mantenido de manera eficiente y efectiva a lo largo del tiempo, teniendo en cuenta factores como el impacto ambiental y la eficiencia energética.

Tecnologías y herramientas

Arquitectura de software

Tecnologías y herramientas

Arquitectura de software

Varían ampliamente según los requisitos del proyecto, la naturaleza del sistema, las preferencias del equipo de desarrollo y las tendencias actuales de la industria. Sin embargo, se pueden categorizar en varios grupos principales que cubren diferentes aspectos del desarrollo de software:

- Lenguajes de programación.
- Bases de datos.
- Frameworks de desarrollo.
- Servidores de aplicaciones y contenedores.
- Plataformas en la nube.
- Herramientas de integración y entrega continua (CI/CD).
- Sistemas de control de versiones.
- Herramientas de monitoreo y logging.
- Herramientas de pruebas.
- Herramientas de desarrollo y diseño de api.
- Herramientas de seguridad.
- Herramientas de gestión de proyectos y colaboración.

Documentación

Arquitectura de software

Documentación

Arquitectura de software

- La documentación adecuada es esencial para describir y comunicar la arquitectura a los miembros del equipo y las partes interesadas. Esto incluye:
 - Justificación de las decisiones tomadas en cuanto a la selección de patrones, estilos, y tecnologías.
 - Descripciones de componentes, conectores e interfaces.
 - Especificación de requisitos funcionales y no funcionales.
 - Diagramas de componentes, despliegue, de secuencia.
 - Modelos de datos.
 - Documentación de APIs.
 - Guías de estilo y convenciones de nomenclatura de código.
 - Manuales de usuario y documentación técnica del sistema.
 - Planes y reporte de pruebas.
 - Guías de despliegue, manuales de mantenimiento.

Consideraciones de evolución y mantenimiento

Arquitectura de software

Consideraciones de evolución y mantenimiento

Arquitectura de software

- La arquitectura debe ser diseñada pensando en la capacidad de evolucionar y mantener el sistema a lo largo del tiempo.
- Flexibilidad y modularidad.
- Documentación actualizada.
- Pruebas regresivas.
- Gestión de dependencias.
- Interoperabilidad.
- Escalabilidad.
- Seguridad.
- Rendimiento.
- Refactorización.
- Análisis de impacto.
- Planificación de la evolución.
- Uso de patrones de diseño.
- Versionado semántico.

Modelos de capas

Unidad 4. Estructura de las aplicaciones web

Saberes teóricos

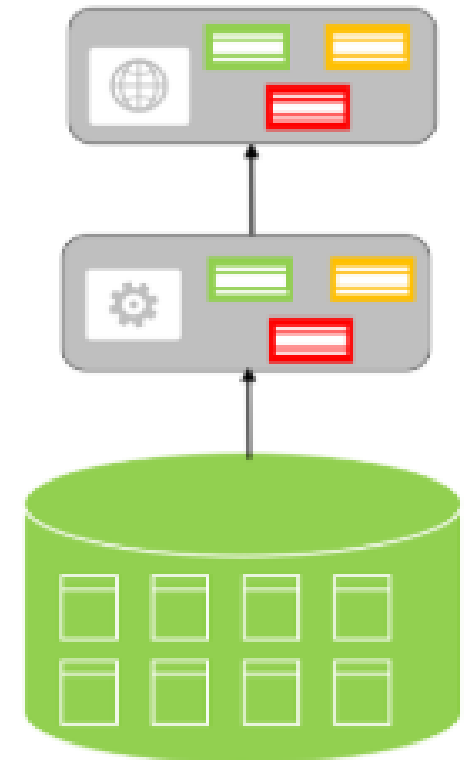
Unidad 4. Desarrollo de Aplicaciones web dinámicas

- **Unidad 4. Desarrollo de Aplicaciones web dinámicas**
 - Taxonomía de las aplicaciones web
 - Marcos de trabajo (frameworks) para el desarrollo de aplicaciones web
 - Aplicaciones web para dispositivos móviles
 - Servidores web
 - Estructura de las aplicaciones web
 - Modelos de capas
 - Separación de estructura, presentación y comportamiento
 - Entrega de contenido multimedia en la web

Arquitectura por capas

Estilo arquitectónico

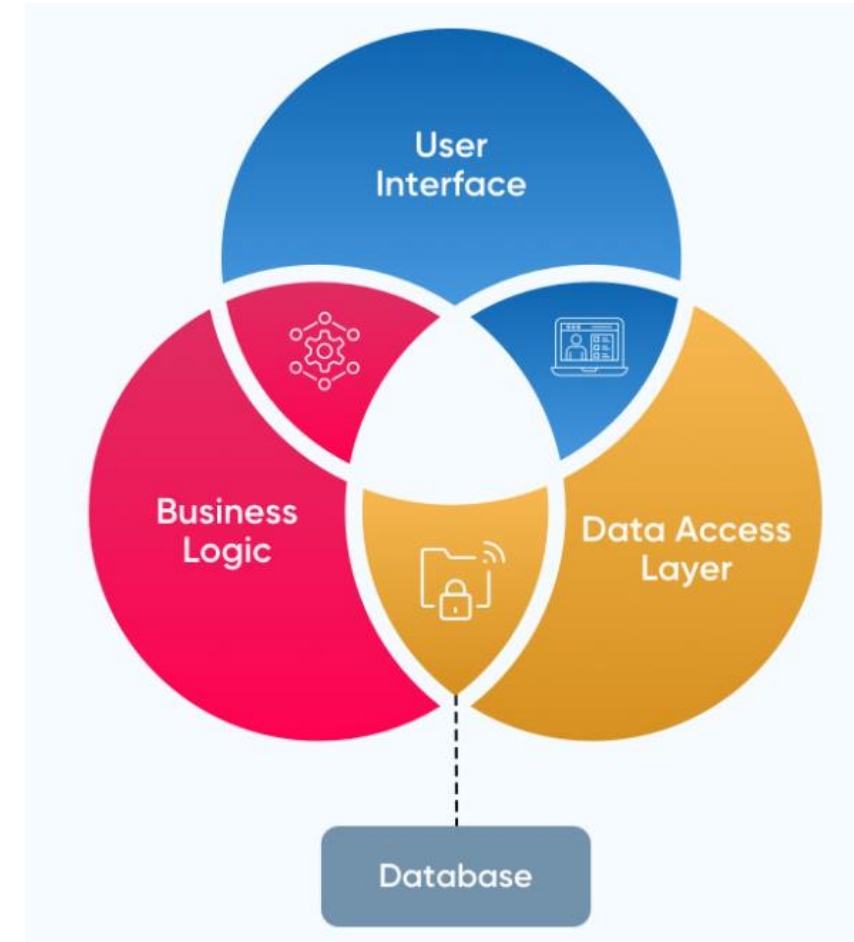
- Una arquitectura por capas amplía la arquitectura cliente-servidor.
- En ella, el servidor se descompone en otros nodos granulares, que desacoplan responsabilidades adicionales del servidor backend, como el procesamiento y la gestión de los datos.
- Estos nodos adicionales se usan para procesar de forma asíncrona las tareas de larga duración y liberar al resto de los nodos del backend para que puedan centrarse en responder a las solicitudes del cliente e interactuar con el almacén de datos.



Componentes de una aplicación web

Modelos de capas

- Lógica de negocio.
 - Parte más importante de la aplicación.
 - Define los procesos que involucran a la aplicación.
 - Conjunto de operaciones requeridas para proveer el servicio.
- Administración de los datos.
 - Manipulación de BD y archivos.
- Interfaz
 - Los usuarios acceden a través de clientes.
 - Toda la funcionalidad accesible a través del navegador.
 - Dirigida por la aplicación.



Modelos de capas

Modelos de capas

- Las aplicaciones web **se modelan** mediante lo que se conoce como modelo de capas.
- Una capa representa un elemento que procesa o trata información.
- Los tipos son:
 - **Modelo de dos capas:** La información atraviesa dos capas entre la interfaz y la administración de los datos.
 - **Modelo de n-capas:** La información atraviesa varias capas, el más habitual es el modelo de **tres capas**.

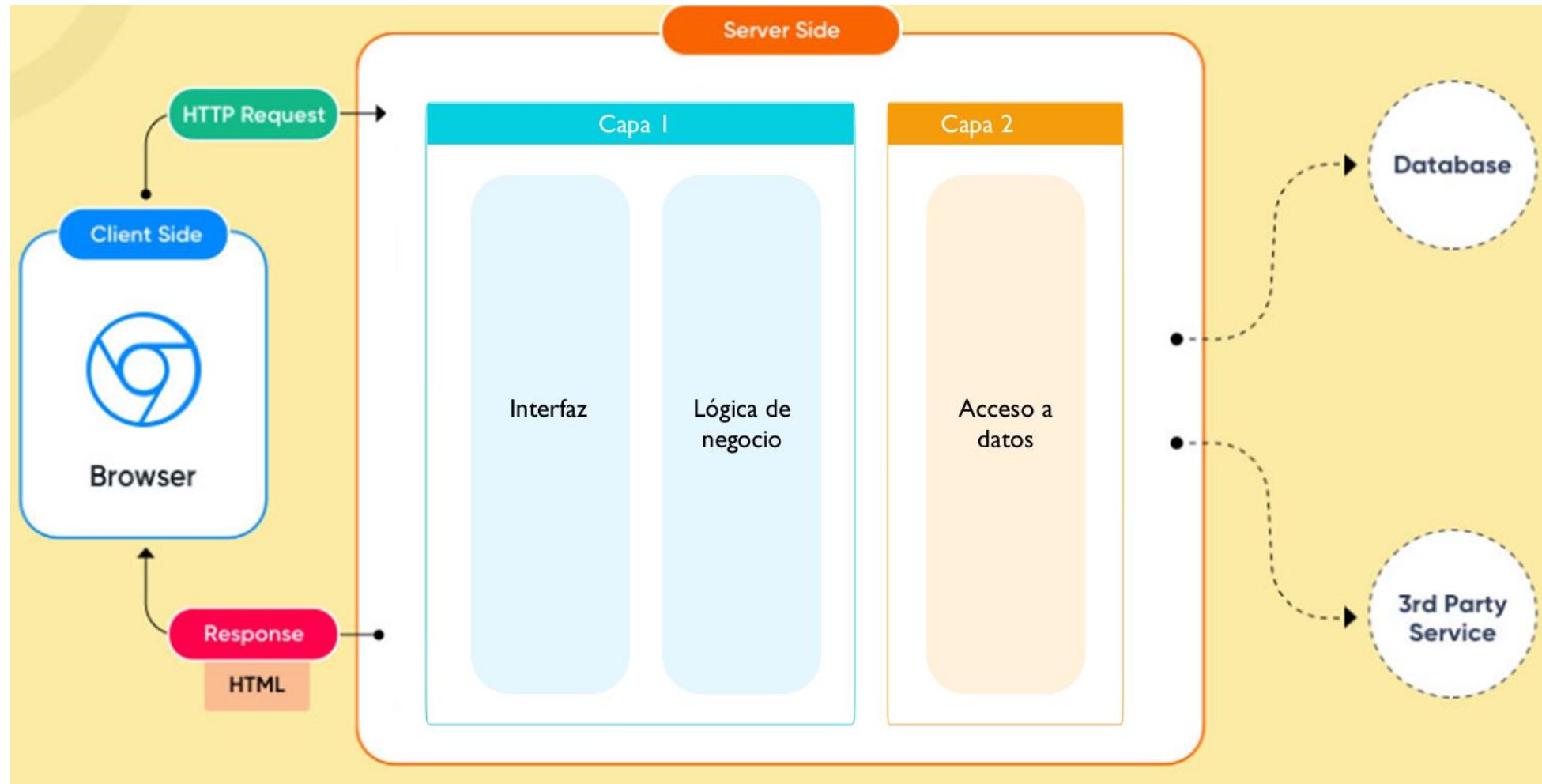
Modelo de dos capas

Modelos de capas

- Gran parte de la aplicación corre en una de las capas.
- Las capas son:
 - **Lógica del negocio y presentación.** La lógica de negocio está inmersa dentro de la aplicación que realiza el interfaz de usuario, en el lado del cliente.
 - **Acceso a datos.** Administra los datos.
- Modelo clásico que se utilizó durante muchos años.
- Los lenguajes como ASP, PHP, JSP fueron los precursores del modelo de dos capas en la web.
- De una “lado” de la página se hacia la interfaz, y del “otro lado” la lógica.
- Generalmente orientado a eventos: OnPageLoad, OnButtonClick.

Modelo de dos capas

Modelos de capas



Modelo de dos capas

Modelos de capas

- Las limitaciones de este modelo son.
 - Es difícilmente escalable.
 - Número de conexiones compartida entre interfaz y lógica de negocio.
 - Alta carga de la red.
 - La flexibilidad es restringida.
 - Complicado trabajar en grupo y/o remotamente al mismo tiempo.

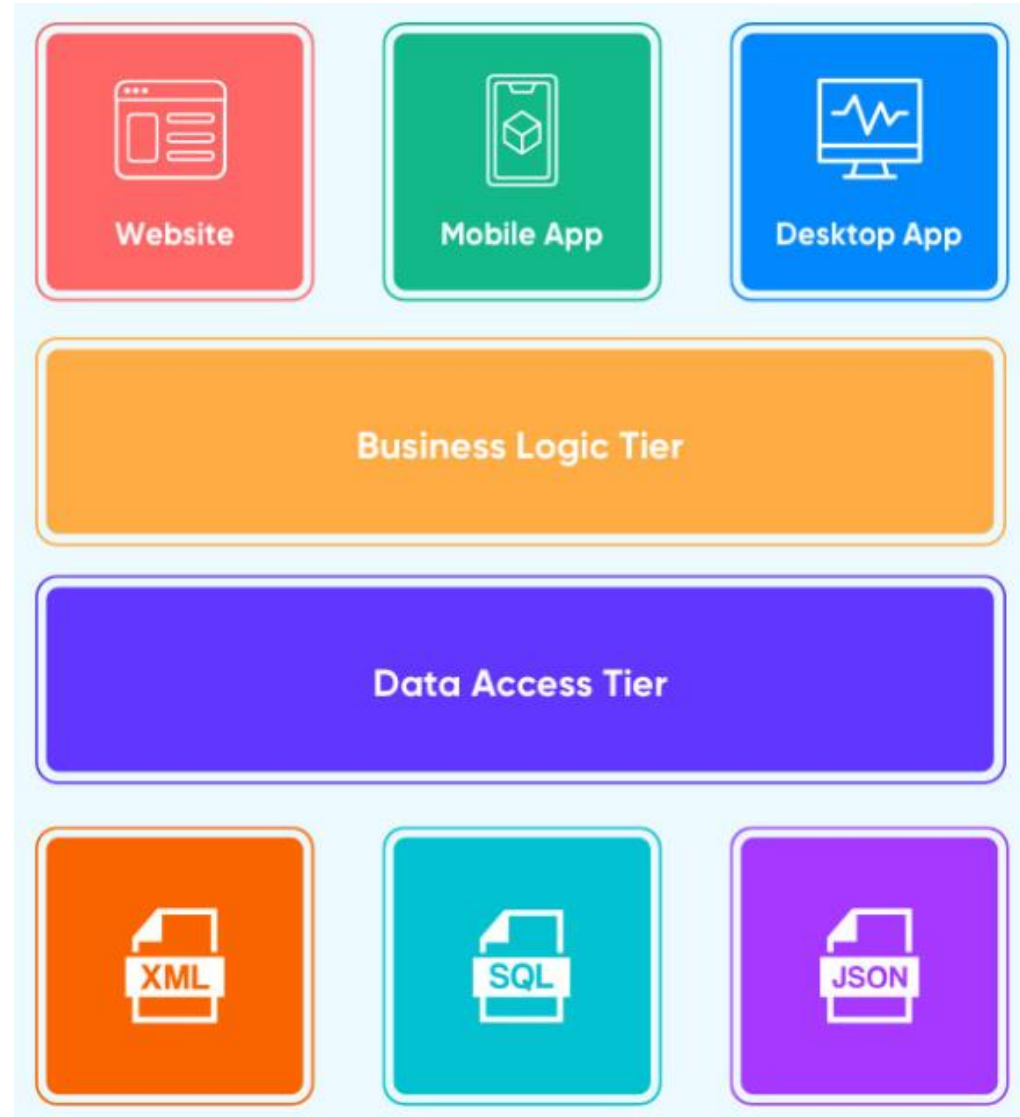
Modelo de n capas

Modelos de capas

- La arquitectura multicapa o de n niveles utiliza capas y niveles en la aplicación para separar y administrar cada nivel de forma independiente.
- Las capas separan las responsabilidades, y las capas superiores acceden servicios en capas inferiores (pero no al revés).
- Esta arquitectura introduce resiliencia y permite que cada servicio escale y funcione de manera óptima.
- En una arquitectura de n niveles, puede haber múltiples niveles para interfaces de usuario para escritorio, dispositivos móviles, etc.
- **Particionar la aplicación lógicamente apoya al desarrollador a crear código fácil de mantener así como dar seguimiento a la actividad de cada capa de forma separada.**

Modelo de n capas

Modelos de capas



Modelo de tres capas

Modelos de capas

- Esta diseñada para superar las limitaciones de las arquitecturas ajustadas al modelo de dos capas, introduce una capa intermedia (la capa de proceso) entre presentación y los datos.
- Los procesos pueden ser manejados de forma separada a la interfaz de usuario y a los datos.
- Esta capa intermedia centraliza la lógica de negocio, haciendo la administración más sencilla.
- Los datos se pueden integrar de múltiples fuentes.
- Las **aplicaciones web actuales** se ajustan a este modelo.

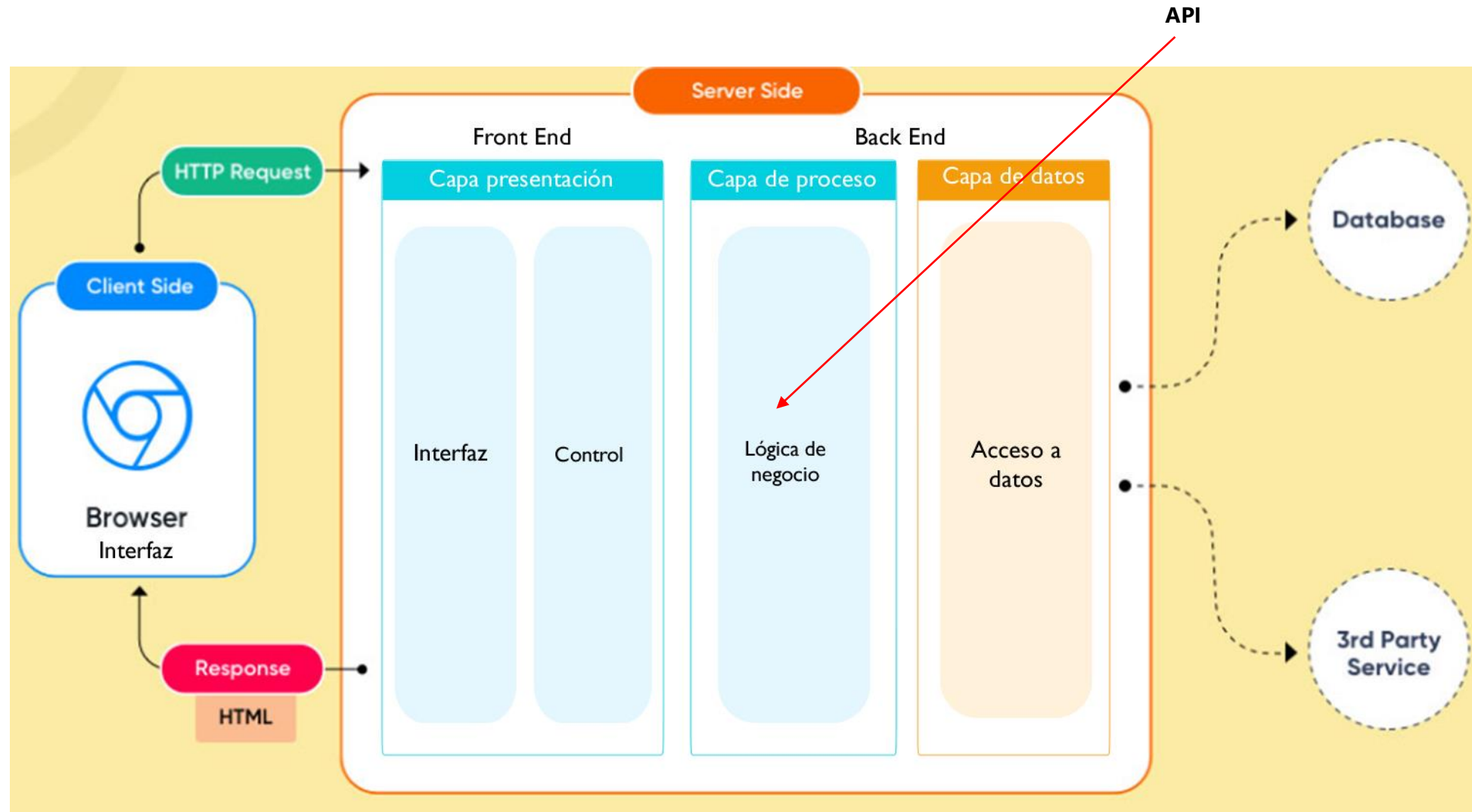
Modelo de tres capas

Modelos de capas

- **Capa de presentación** (parte en el cliente y parte en el servidor web)
 - Recoge la información del usuario y la envía al servidor (cliente).
 - Manda información a la capa de proceso para su procesamiento.
 - Recibe los resultados de la capa de proceso.
 - Generan la presentación.
 - Visualizan la presentación al usuario (cliente).
- **Capa de proceso o negocio** (servidor web)
 - Recibe la entrada de datos de la capa de presentación.
 - Interactúa con la capa de datos para realizar operaciones.
 - Manda los resultados procesados a la capa de presentación.
- **Capa de datos** (servidor de datos)
 - Almacena los datos.
 - Recupera datos.
 - Mantiene y asegura la integridad de los datos.

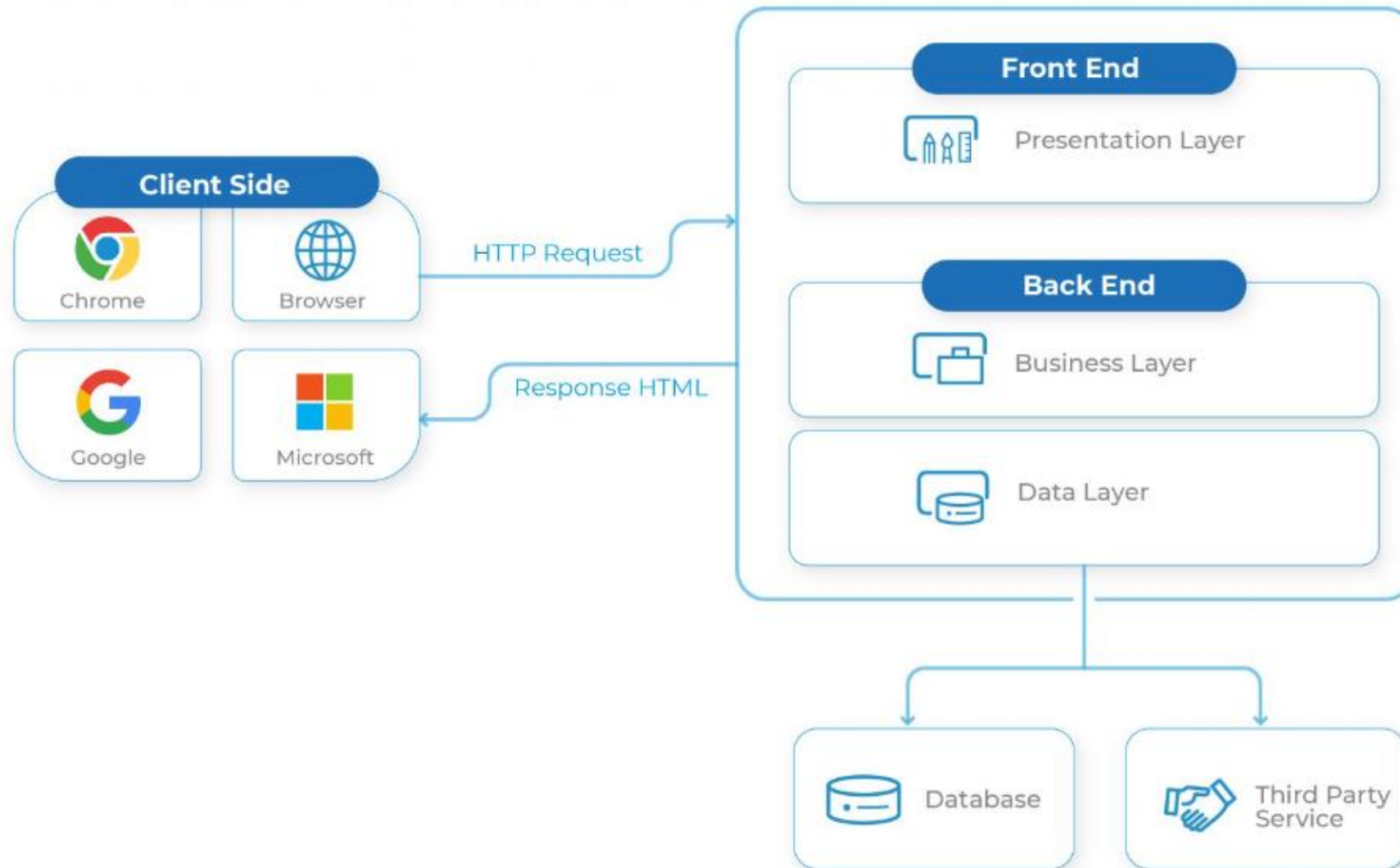
Modelo de tres capas

Modelos de capas



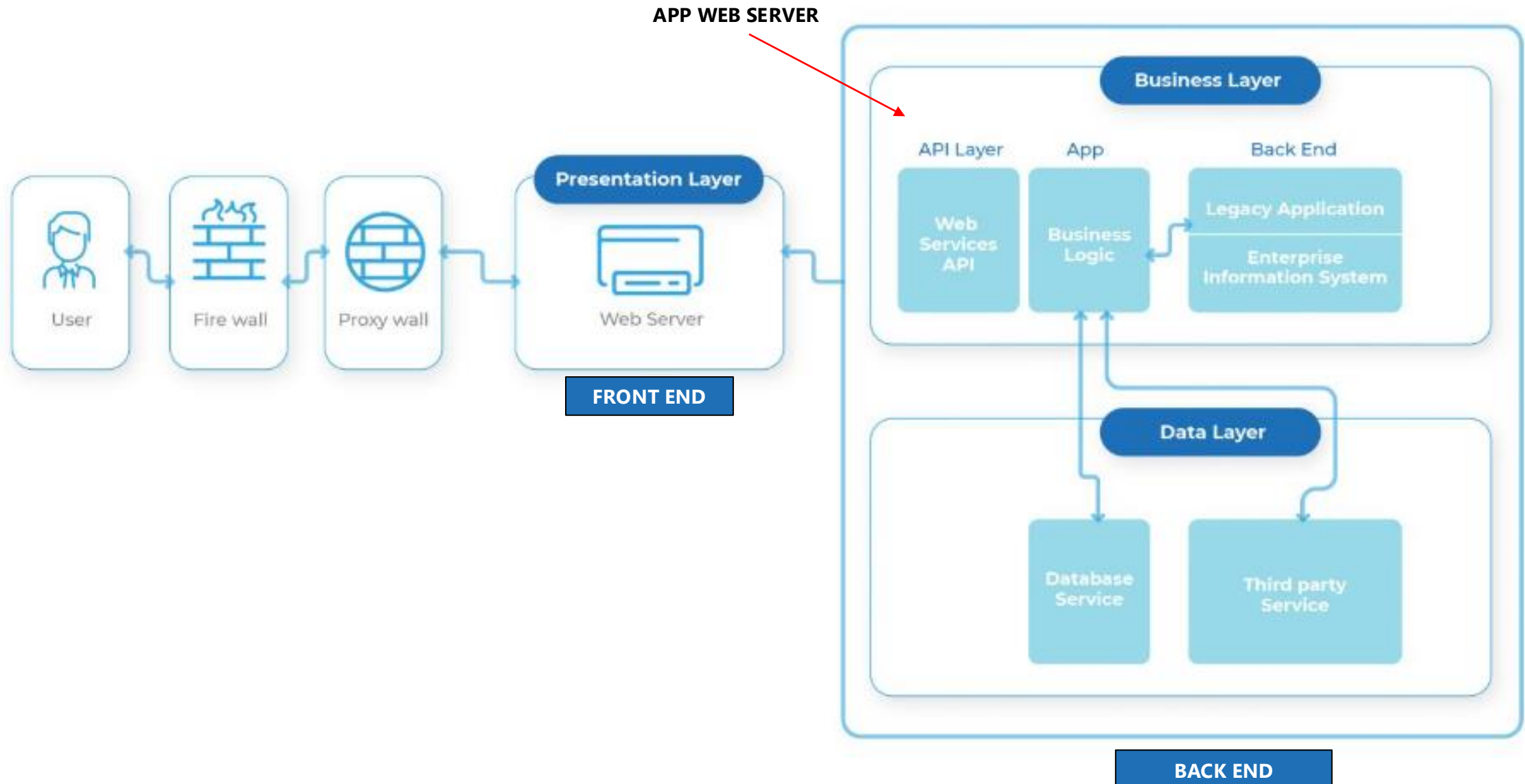
Modelo de tres capas

Modelos de capas



Modelo de tres capas clásico

Modelos de capas



Modelo de tres capas moderno: Front End

Modelos de capas



Modelo de tres capas moderno: Back End

Modelos de capas



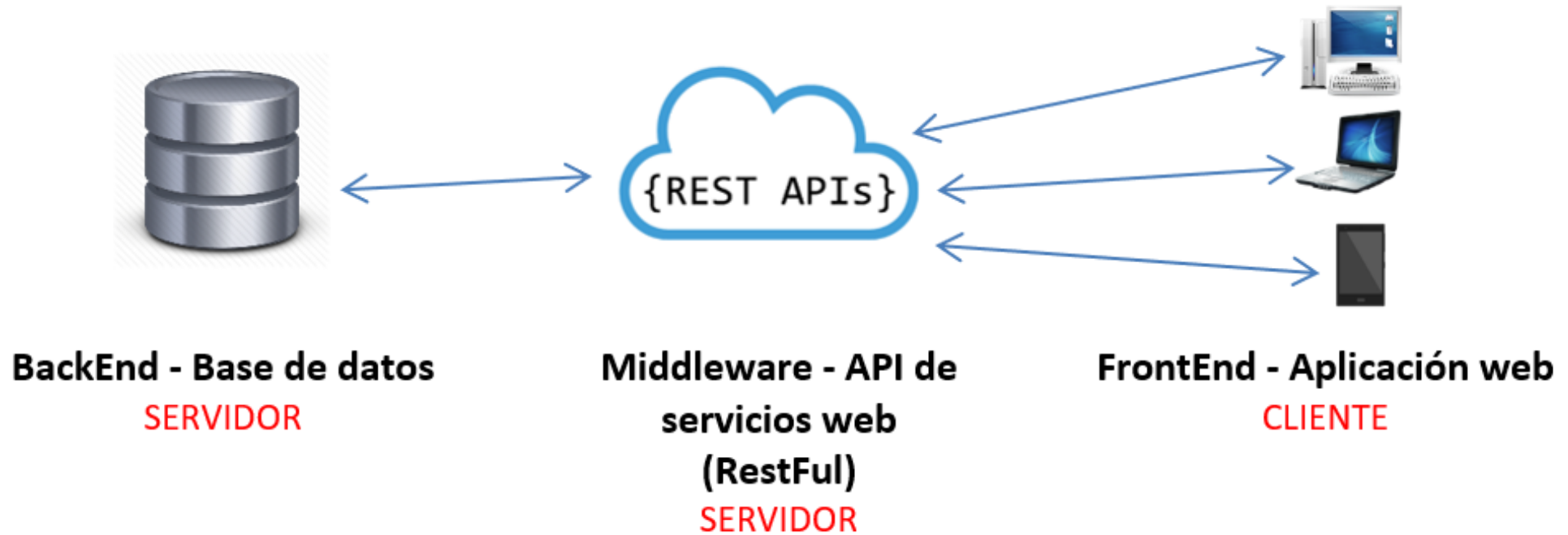
Modelo de tres capas moderno: Application Programming Interface (API)

Modelos de capas

- La interfaz de programación de aplicaciones (API) no es una tecnología, es un concepto que permite a los desarrolladores acceder a ciertos datos y funciones de un software.
- Es un mediador que permite que las aplicaciones se comuniquen entre sí.
- Comprende protocolos, herramientas y definiciones de subrutinas necesarias para crear aplicaciones.
 - **RESTful**: Representational State Transfer usa formato JSON ligero. Es altamente escalable, confiable y ofrece un rendimiento rápido, lo que la convierte en la API más popular.
 - **SOAP**: Simple Object Access Protocol utiliza XML para la transmisión de datos. Requiere más ancho de banda y seguridad avanzada.
 - **XML-RPC**: Remote Procedure Calls utilizan un formato XML específico para la transmisión de datos
 - **JSON-RPC**: Utiliza el formato JSON para la transmisión de datos.

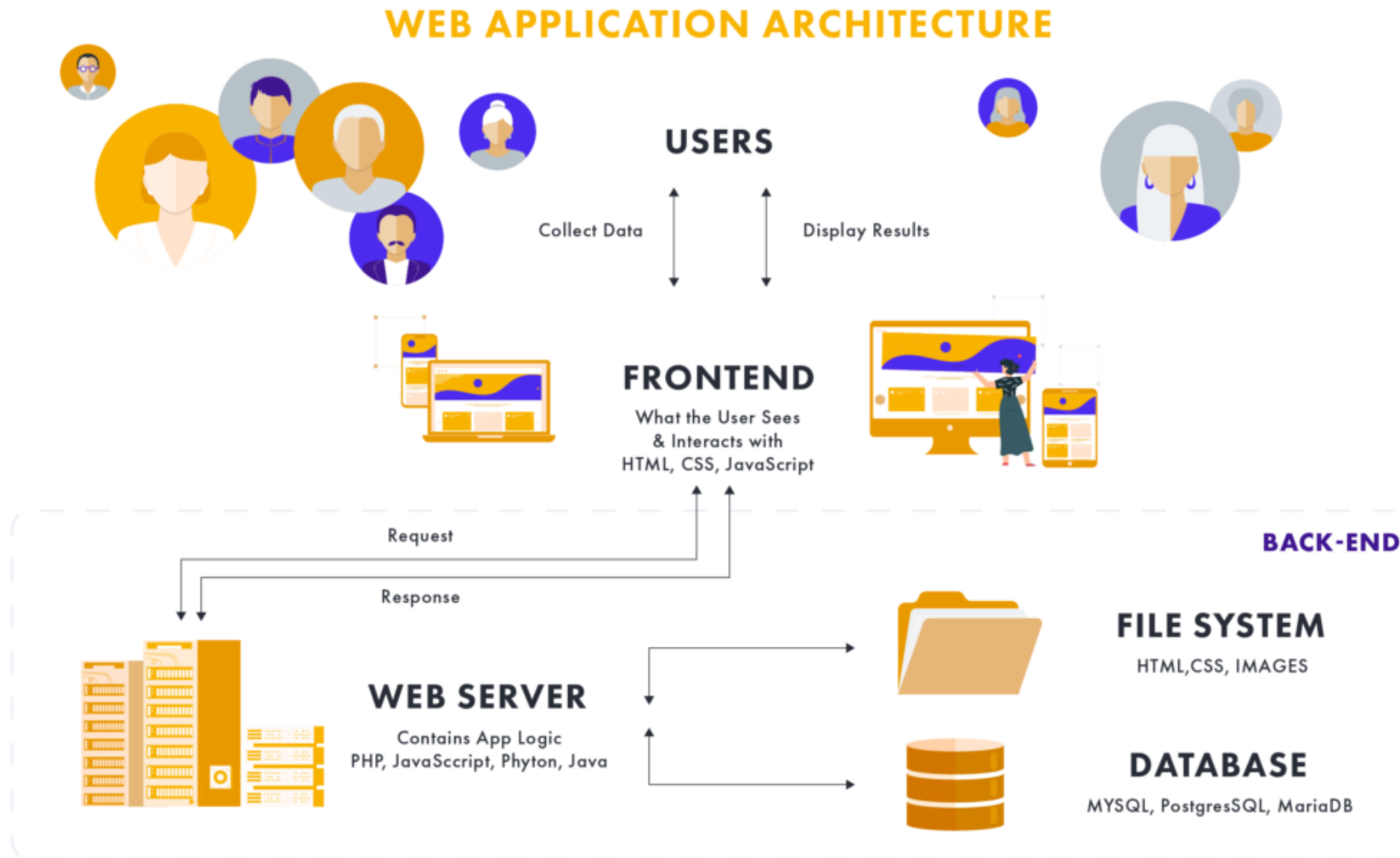
Modelo de tres capas moderno

Modelos de capas



Arquitectura de aplicaciones web

Unidad II. Frameworks de desarrollo Web



Patrones de diseño de aplicaciones en capas

Arquitectura de software

Algunos ejemplos son:

- Modelo-Vista-Presentador (MVP).
- Modelo-Vista-ViewModel (MVVM).
- Modelo-Vista-Controlador (MVC).

Separación de estructura, presentación y comportamiento

Unidad 4. Estructura de las aplicaciones web

Saberes teóricos

Unidad 4. Desarrollo de Aplicaciones web dinámicas

- **Unidad 4. Desarrollo de Aplicaciones web dinámicas**
 - Taxonomía de las aplicaciones web
 - Marcos de trabajo (frameworks) para el desarrollo de aplicaciones web
 - Aplicaciones web para dispositivos móviles
 - Servidores web
 - Estructura de las aplicaciones web
 - Modelos de capas
 - Separación de estructura, presentación y comportamiento
 - Entrega de contenido multimedia en la web

Introducción

Separación de estructura, presentación y comportamiento

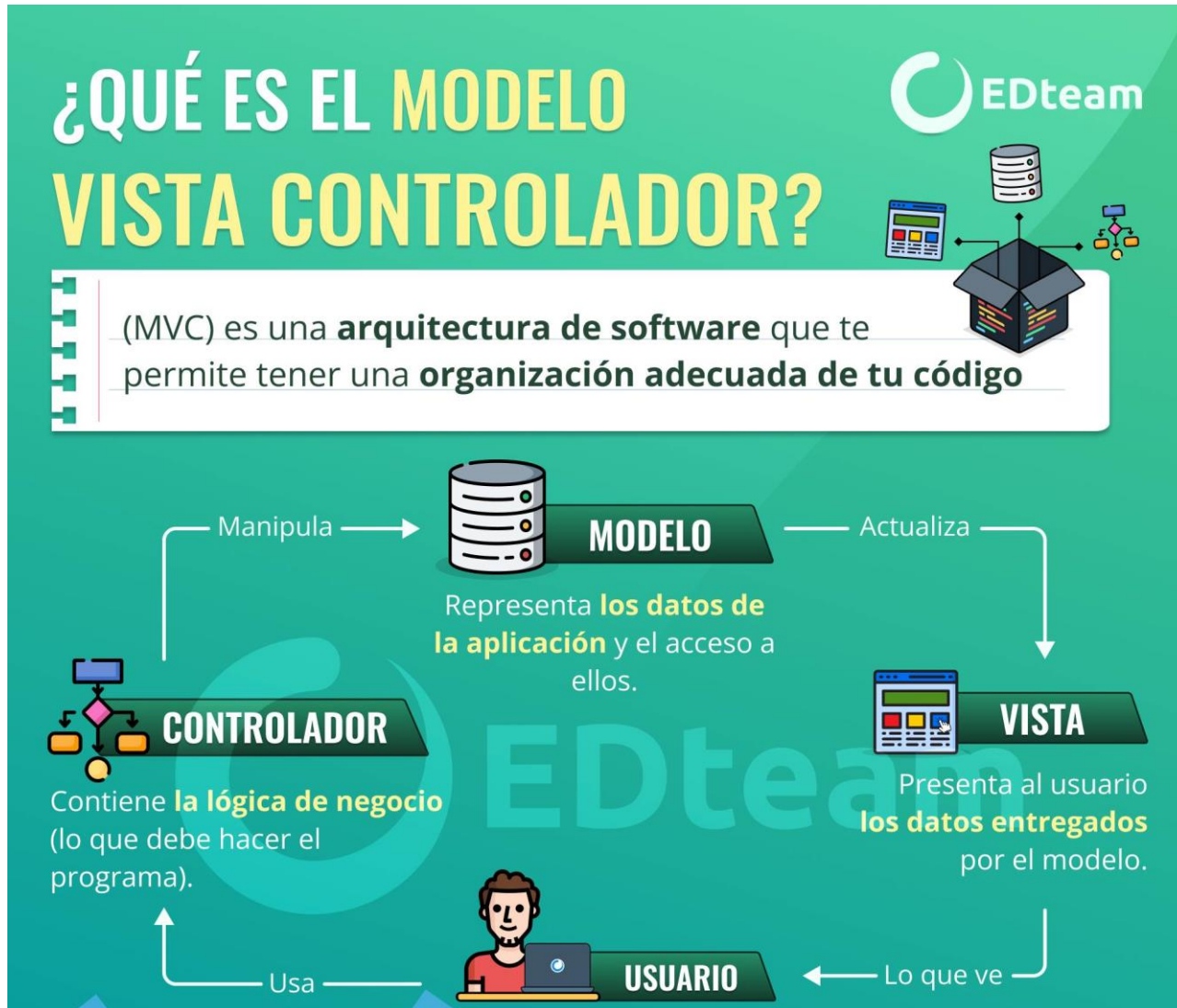
- Los **patrones arquitectónicos** son la columna vertebral de una aplicación escalable y bien diseñada. Proporcionan un modelo reutilizable para resolver problemas recurrentes en la arquitectura de software, facilitando la separación de actividades y mejorando la mantenibilidad y reutilización del código.
- Tres de los patrones arquitectónicos más populares son:
 - **Model-View-Controller** (*MVC*).
 - **Model-View-Presenter** (*MVP*).
 - **Model-View-ViewModel** (*MVVM*).
- Cada uno tiene ventajas únicas y ofrece soluciones para diferentes proyectos y requisitos.
- Comprender estos patrones ayuda a los desarrolladores a diseñar software de manera más efectiva, crear mejores soluciones y garantizar que sus proyectos puedan crecer y adaptarse a los requisitos cambiantes.

Modelo Vista Controlador (MVC)

Patrones de diseño MVC, MVP, MVVM

Modelo Vista Controlador (MVC)

Patrones de diseño MVC, MVP, MVVM



Estructura, presentación y comportamiento

Modelo, vista y controlador



Trygve Reenskaug

Científico de la computación noruego

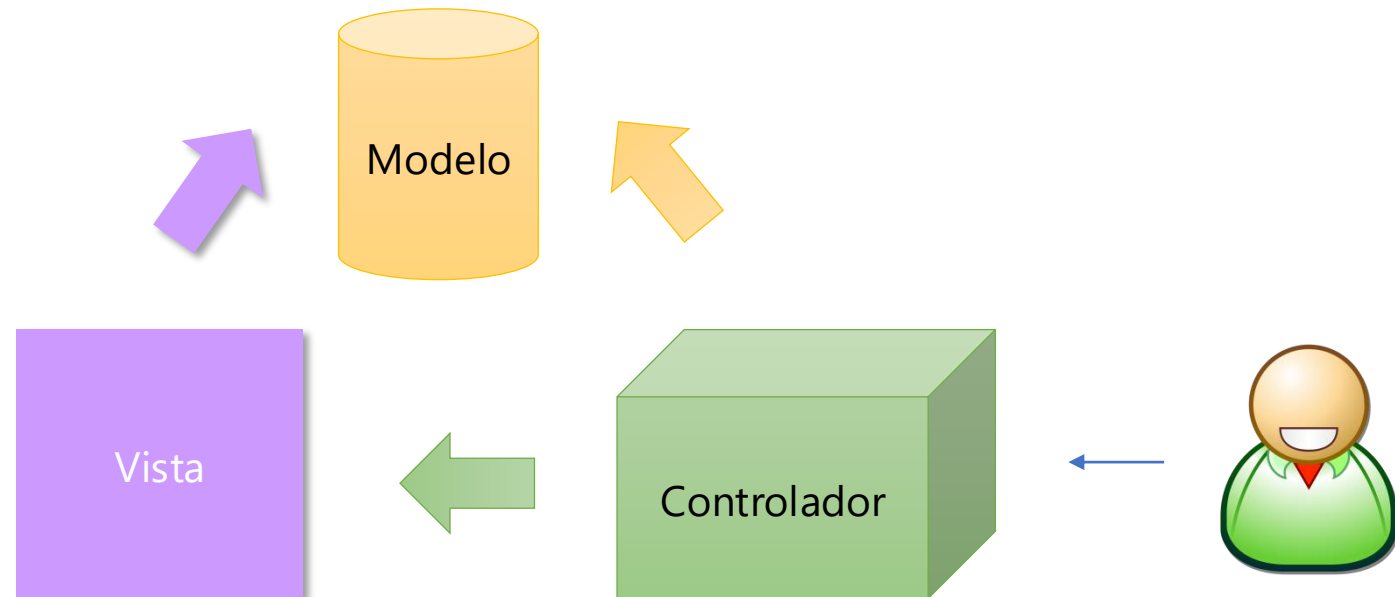


Traducción del inglés - Trygve Mikkjel Heyerdahl Reenskaug es un informático noruego y profesor emérito de la Universidad de Oslo. Formuló el patrón modelo-vista-controlador para el diseño de software de interfaz gráfica de usuario en 1979 mientras visitaba el Centro de Investigación Xerox Palo Alto. [Wikipedia \(Inglés\)](#)

¿Qué es MVC?

Modelo Vista Controlador (MVC)

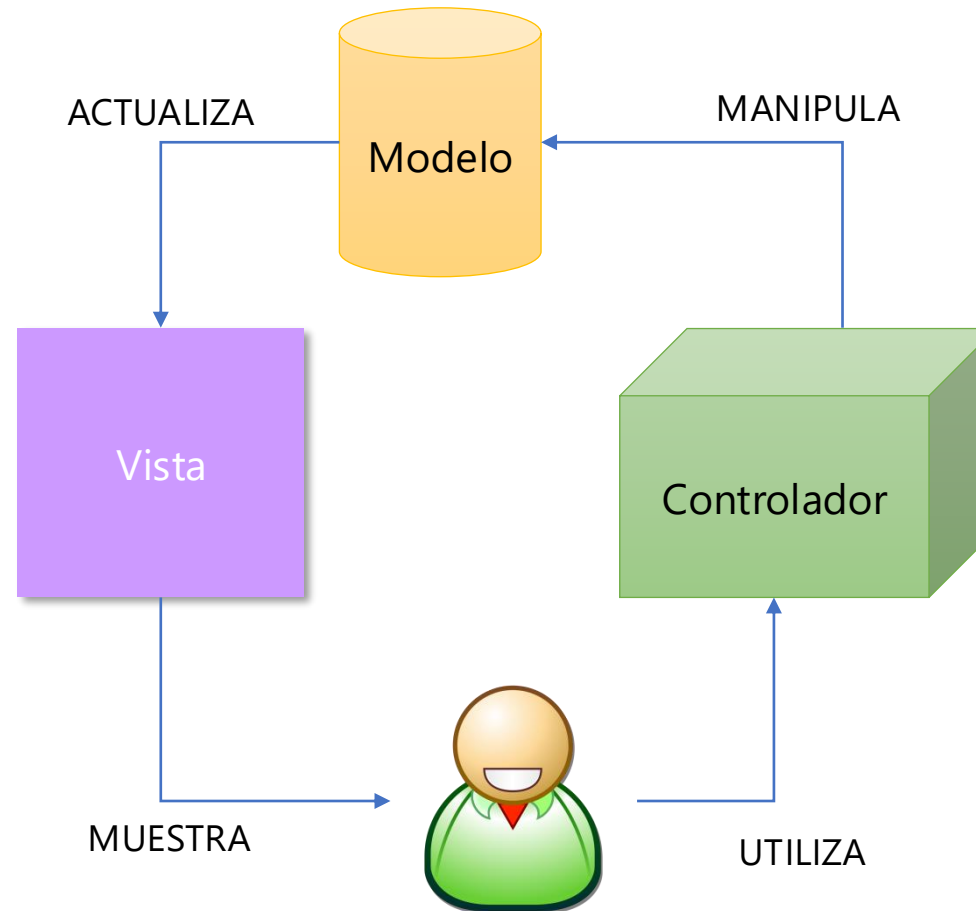
- Patrón de arquitectura de software que separa el modelo, la interfaz de usuario y el control de la lógica de una aplicación en tres distintos componentes:
 - **Modelo:** Representa los datos y la lógica empresarial de la aplicación.
 - **Vista:** Representa la interfaz de usuario (UI) y la capa de presentación de la aplicación.
 - **Controlador:** Actúa como intermediario entre el modelo y la vista. El controlador recibe información del usuario desde la vista, la procesa y actualiza el modelo.



Componentes de MVC

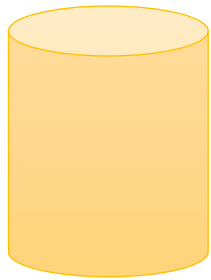
Modelo Vista Controlador (MVC)

- MVC propone la construcción de tres distintos componentes:
 1. Modelo
 2. Vista
 3. Controlador



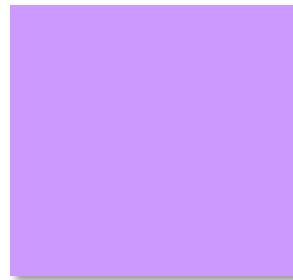
Patrón de MVC

Modelo Vista Controlador (MVC)



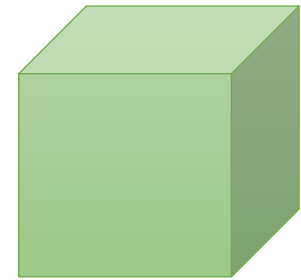
El modelo

Representación de los datos
Lógica del negocio
Obtiene y almacena datos en un almacenamiento persistente (Base de Datos)



La vista

Interfaz de usuario a partir del modelo
Elementos de interacción (HTML, XML)

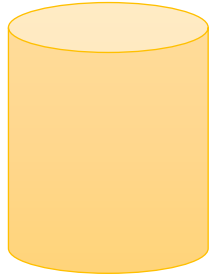


El controlador

Maneja la interacción con el usuario e invoca cambios al Modelo y Vistas.
Intermediario entre el Modelo y la Vista.

Jerarquía de dependencia en MVC

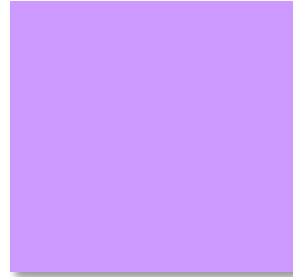
Modelo Vista Controlador (MVC)



El modelo

El Modelo solo conoce sobre el mismo.

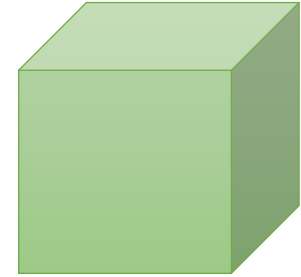
Esto quiere decir, que el código fuente del Modelo no hace referencias ni a la Vista o al Controlador.



La vista

La Vista si conoce al Modelo.

Preguntará al Modelo sobre su estado para saber que mostrar. De esta manera, la Vista puede desplegar algo que esta basado en lo que el Modelo ha hecho. La Vista no sabe nada del Controlador.



El controlador

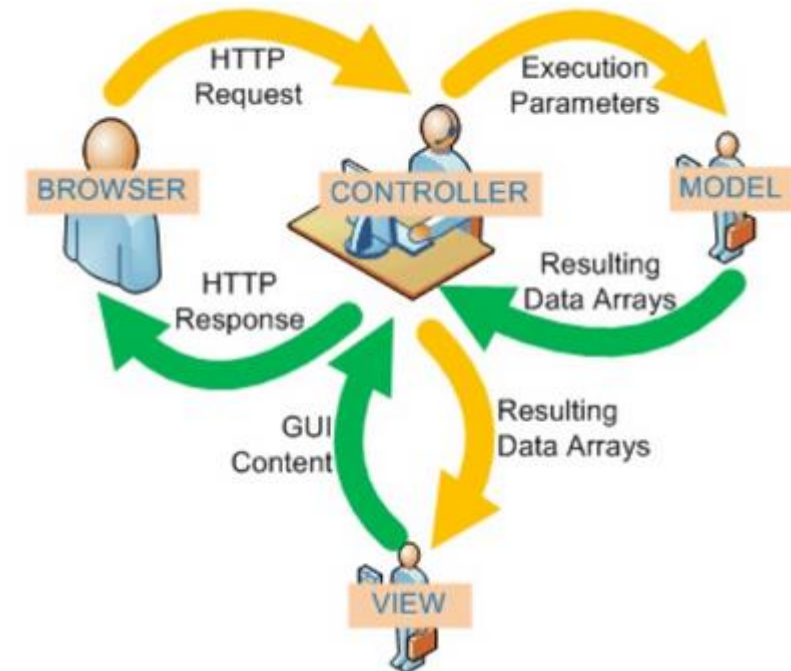
Conoce al Modelo y a la Vista.

Si el usuario realiza una acción, el Controlador sabe que función en el Modelo llamar y también sabe que Vista mostrar al usuario.

Flujo de MVC

Modelo Vista Controlador (MVC)

1. El usuario realiza una acción en la interfaz (ej. presiona un botón)
2. El Controlador toma el evento o acción de entrada
3. El Controlador notifica la acción al Modelo, la cuál pudiera involucrar un cambio en el Modelo (ej. edición de datos).
4. Esto genera una nueva Vista que se envía al usuario. La Vista toma los datos del Modelo (ej. lista de usuarios).
5. La interfaz de usuario espera por otra interacción de usuario, lo que inicia un nuevo ciclo.



Beneficios de MVC

Modelo Vista Controlador (MVC)

- **Fácil** de manejar la complejidad.
- Desarrollo de aplicaciones más **rápido**.
- **Reusabilidad** del código.
- Desarrollo en **paralelo**.
- Facilita la **presentación** de información de diferentes maneras donde el código de la aplicación no se ve afectado.
- Ideal para sistemas **grandes** y distribuidos.
- Gran control sobre el **comportamiento** de la aplicación.
- Uso principalmente en **aplicaciones web**.

Desventajas de MVC

Modelo Vista Controlador (MVC)

- El controlador puede convertirse en un cuello de botella para aplicaciones complejas con muchas interacciones del usuario.
- Puede ser difícil de implementar para aplicaciones con requisitos de estado o interacción complicados.

PHP

Desarrollo de Aplicaciones Web Dinámicas

PHP (Hypertext Preprocessor) es un lenguaje de programación del lado del servidor.

Se utiliza para:

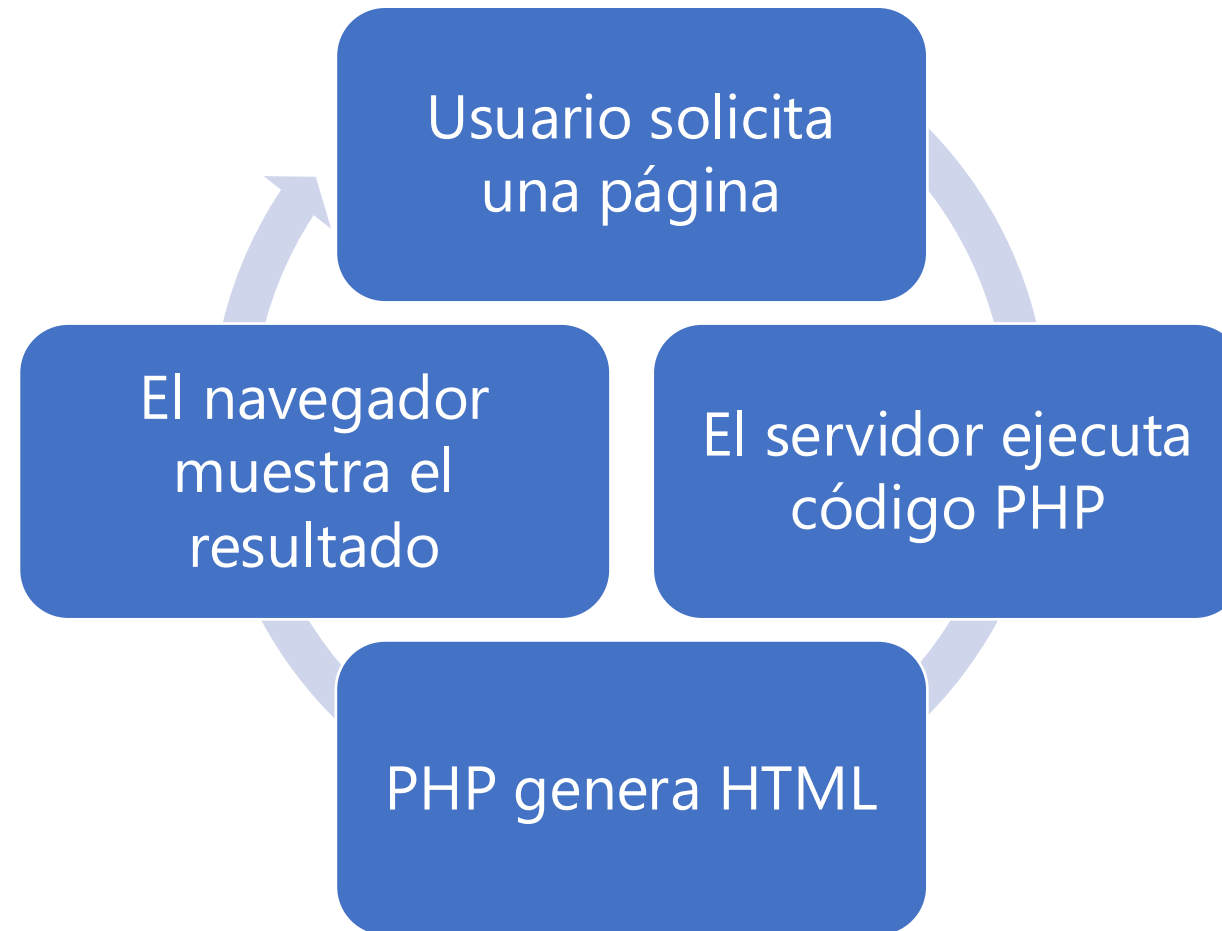
- Crear aplicaciones web dinámicas
- Procesar formularios
- Interactuar con bases de datos
- Generar contenido HTML
- Fue creado por Rasmus Lerdorf en 1994, inicialmente para scripts para páginas personales y posteriormente evoluciona a lenguaje web completo

Hoy en día es:

- Ampliamente usado en la web
- Base de sistemas como WordPress

¿Cómo funciona PHP?

Desarrollo de Aplicaciones Web Dinámicas



El navegador **nunca ve el código PHP**, solo el HTML generado.

PHP

Desarrollo de Aplicaciones Web Dinámicas

Cliente (Frontend)	Servidor (Backend)
HTML, CSS, JS	PHP
Se ejecuta en navegador	Se ejecuta en servidor
Interfaz	Lógica

PHP. Sintaxis básica

Desarrollo de Aplicaciones Web Dinámicas

Las instrucciones de PHP se insertan de la siguiente forma:

```
<?php
    // Código PHP aquí
?>
```

Por ejemplo:

```
<?php
    echo "Hola mundo";
?>
```

Variables:

```
<?php
    $nombre = "Erika";
    $edad = 46;

    echo "Hola, soy " . $nombre;
?>
```

PHP. Sintaxis básica

Desarrollo de Aplicaciones Web Dinámicas

Estructuras de control

```
<?php
    $edad = 18;

    if ($edad >= 18) {
        echo "Mayor de edad";
    } else {
        echo "Menor de edad";
    }
?>
```

Arreglos y ciclos

```
<?php
    $colores = ["rojo", "verde", "azul"];

    foreach ($colores as $color) {
        echo $color . "<br>";
    }
?>
```

PHP. Sintaxis básica

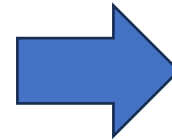
Desarrollo de Aplicaciones Web Dinámicas

PHP + HTML

```
<h1>Bienvenido</h1>
```

```
<?php  
    echo "<p>Contenido generado con PHP</p>";  
?>
```

```
<?php  
    $nombre = "Estudiante";  
  
    echo "<h2>Bienvenido, $nombre</h2>";  
?>
```



```
<h2>Bienvenido, Estudiante</h2>
```

PHP

Desarrollo de Aplicaciones Web Dinámicas

En conclusión:

PHP es un lenguaje del lado del servidor

Genera HTML dinámicamente

Se integra con bases de datos

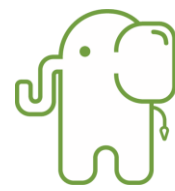
Es clave para aplicaciones web

Lo emplearemos en la siguiente práctica para:

- Implementar backend
- Aplicar arquitectura MVC
- Procesar formularios
- Guardar información en mysql

Slim (PHP)

Frameworks de backend



Slim Framework

- Lanzado en **2005**. Es un microframework ideal para pequeñas aplicaciones web y APIs. Slim enfoca su prioridad en la seguridad y velocidad de respuesta, a lo que contribuye su simplicidad y peso ligero. Slim ofrece terminar tu trabajo con rapidez y calidad.
- **Características**
 - Permite escribir rápidamente aplicaciones web y APIs.
 - Aporta un sistema de rutas.
 - Uso de middlewares para interceptar el flujo normal del proyecto y aportar funcionalidad.
 - Ofrece inyección de dependencias.
 - Slim es un framework de software libre y código abierto.
- **¿Cuándo utilizar?**
 - Aplicaciones pequeñas de manera rápida, flexible y sencilla.

Actividad en clase

Modelo Vista Controlador (MVC)

- **Nombre:** Aplicación MVC con PHP.
- **Tema:** Separación de estructura, presentación y comportamiento.
- **Instrucciones:**
 - Realiza la práctica indicada en la actividad de Eminus.
 - Coloca el resultado de la actividad como se indica.





Preguntas